

Digital Design & Computer Arch.

Lecture 9a: ISA & Microarchitecture

Prof. Onur Mutlu

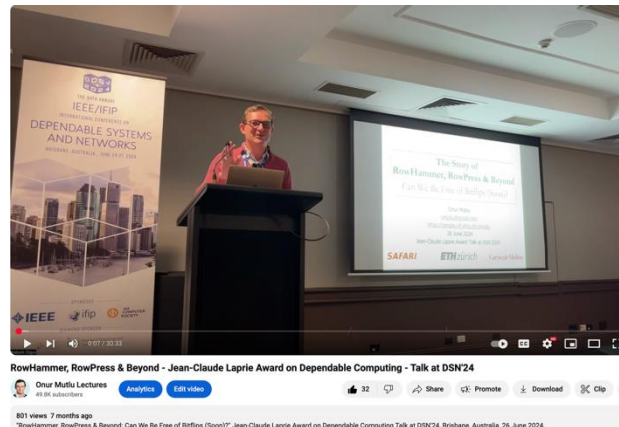
ETH Zürich

Spring 2026

19 March 2026

Extra Credit Assignment: Talk Analysis

- The Story of RowHammer, RowPress & Beyond
- **Watch and analyze this short lecture (30 minutes)**
 - ❑ <https://www.youtube.com/watch?v=U1EcqXlclKU> (June 2024)



- **Assignment – for 1% extra credit**
 - ❑ **Write a good 1-page individualized summary (no AI use)**
 - What are your key takeaways? What did you learn?
 - What surprised you about the content presented? What excited you?
 - What do you think solutions should be like?
 - Submit your summary to Moodle – **deadline March 21**

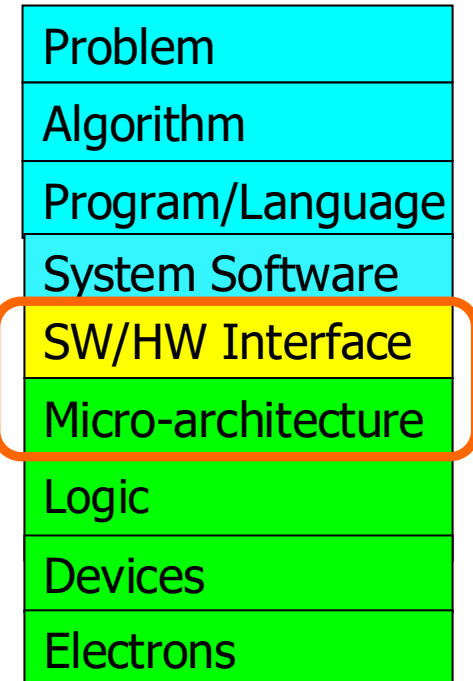
Agenda for Today & Next Few Lectures

- The von Neumann model
- LC-3: An example of von Neumann machine
- LC-3 and MIPS Instruction Set Architectures
- **LC-3 and MIPS assembly and programming**
- Introduction to microarchitecture and single-cycle microarchitecture
- Multi-cycle microarchitecture

Problem
Algorithm
Program/Language
System Software
SW/HW Interface
Micro-architecture
Logic
Devices
Electrons

What Have We Been Learning?

- Basic elements of a computer & the von Neumann model
 - LC-3: An example von Neumann machine
- Instruction Set Architectures: LC-3 and MIPS
 - Operate instructions
 - Data movement instructions
 - Control instructions
- Instruction formats
- Addressing modes



Readings

■ Last week

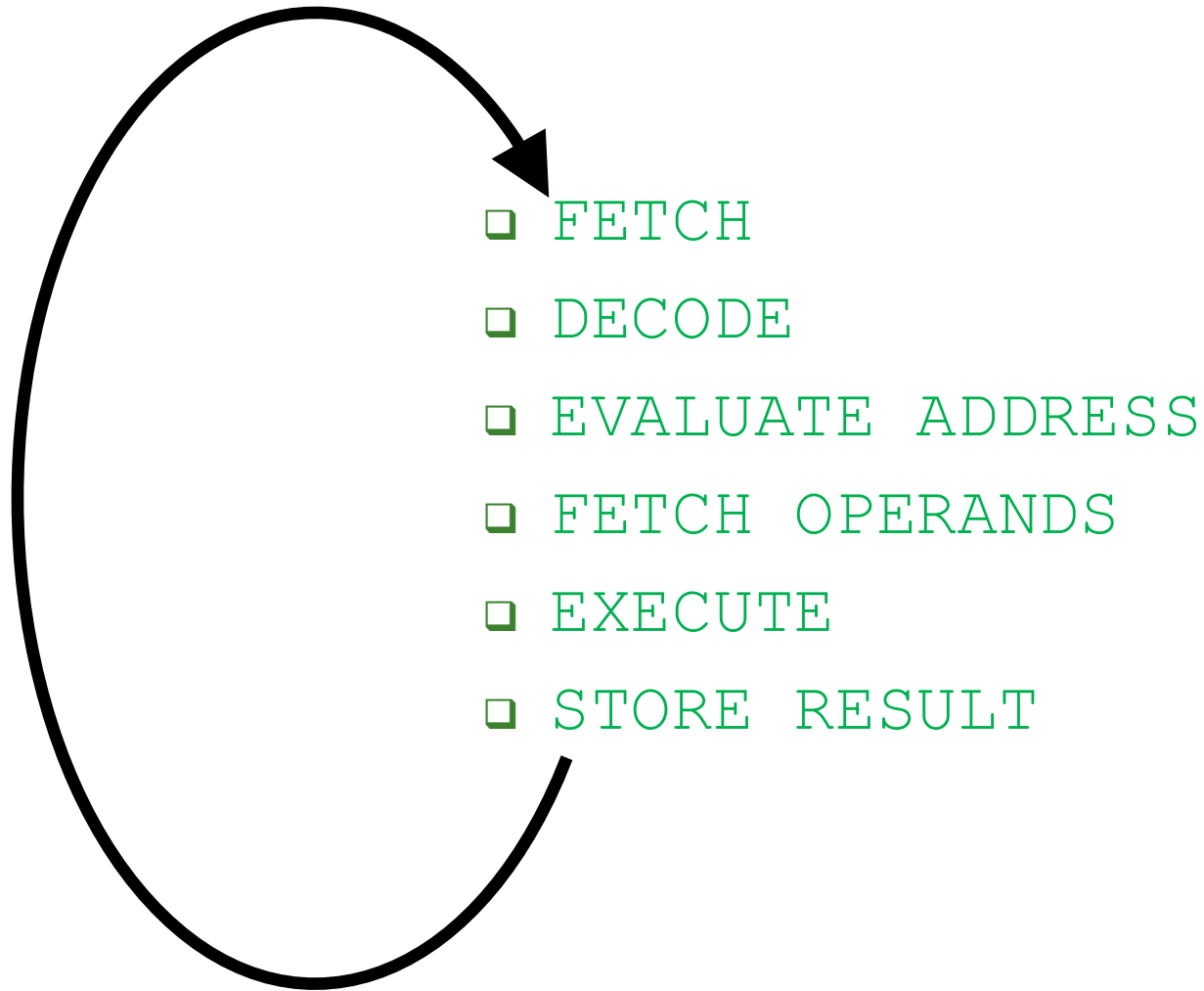
- Von Neumann Model, ISA, LC-3, and MIPS
 - P&P, Chapters 4, 5 (we will follow these today & tomorrow)
 - H&H, Chapter 6 (until 6.5)
 - P&P, Appendices A and C (ISA and microarchitecture of LC-3)
 - H&H, Appendix B (MIPS instructions)
- Programming
 - P&P, Chapter 6 (we will follow this tomorrow)
- **Recommended:** H&H Chapter 5, especially 5.1, 5.2, 5.4, 5.5

■ This week

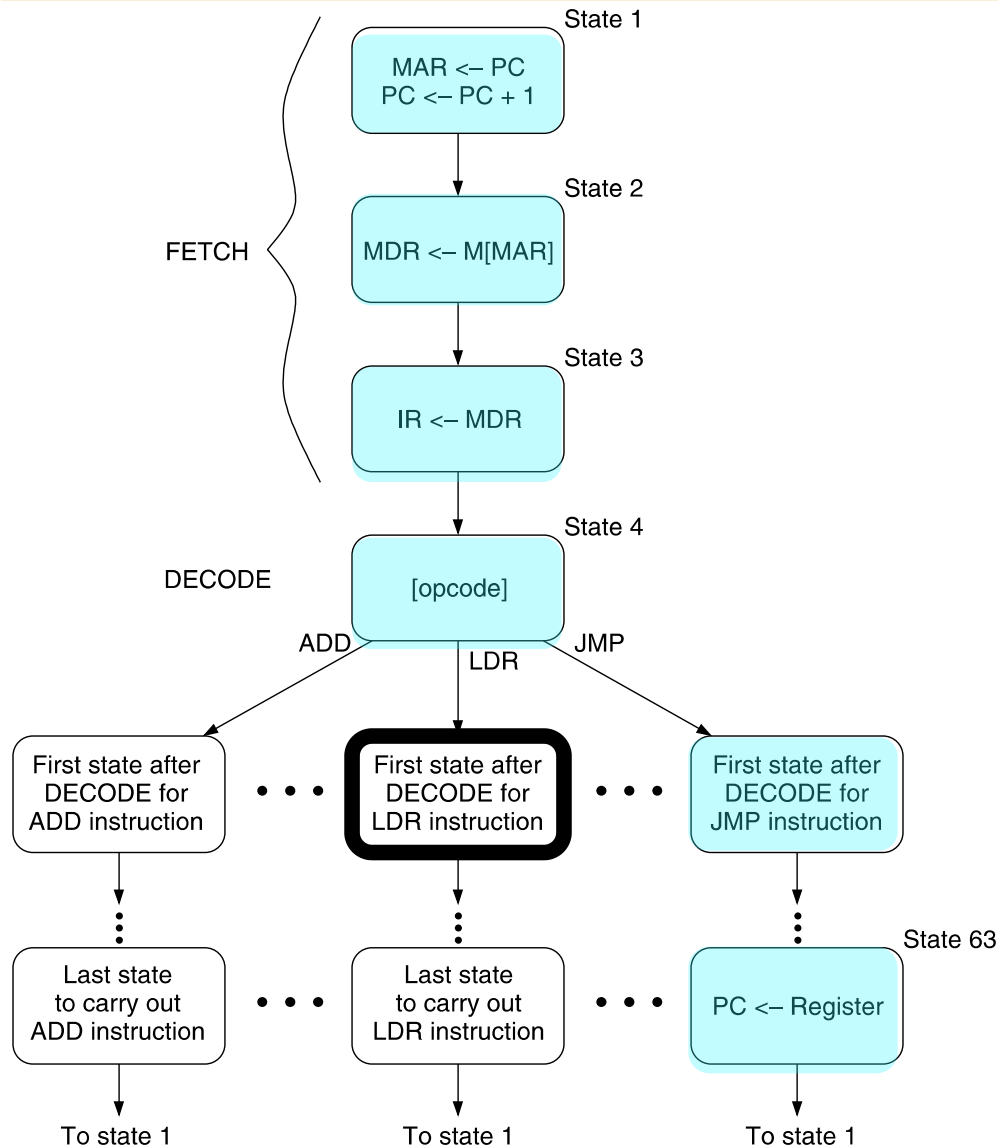
- Introduction to microarchitecture and single-cycle microarchitecture
 - H&H, Chapter 7.1-7.3
 - P&P, Appendices A and C
- Multi-cycle microarchitecture
 - H&H, Chapter 7.4
 - P&P, Appendices A and C

Instruction (Processing) Cycle

Recall: The Instruction (Processing) Cycle



Recall: Control of the Instruction Cycle



- State 1
 - The FSM asserts GatePC and LD.MAR
 - It selects input (+1) in PCMUX and asserts LD.PC
- State 2
 - MDR is loaded with the instruction
- State 3
 - The FSM asserts GateMDR and LD.IR
- State 4
 - The FSM goes to next state depending on opcode
- State 63
 - JMP loads register into PC
- Full state diagram in Patt&Pattel, Appendix C

Figure 4.4 An abbreviated state diagram of the LC-3

This is an FSM Controlling the LC-3 Processor

Recall: Full State Machine for LC-3b

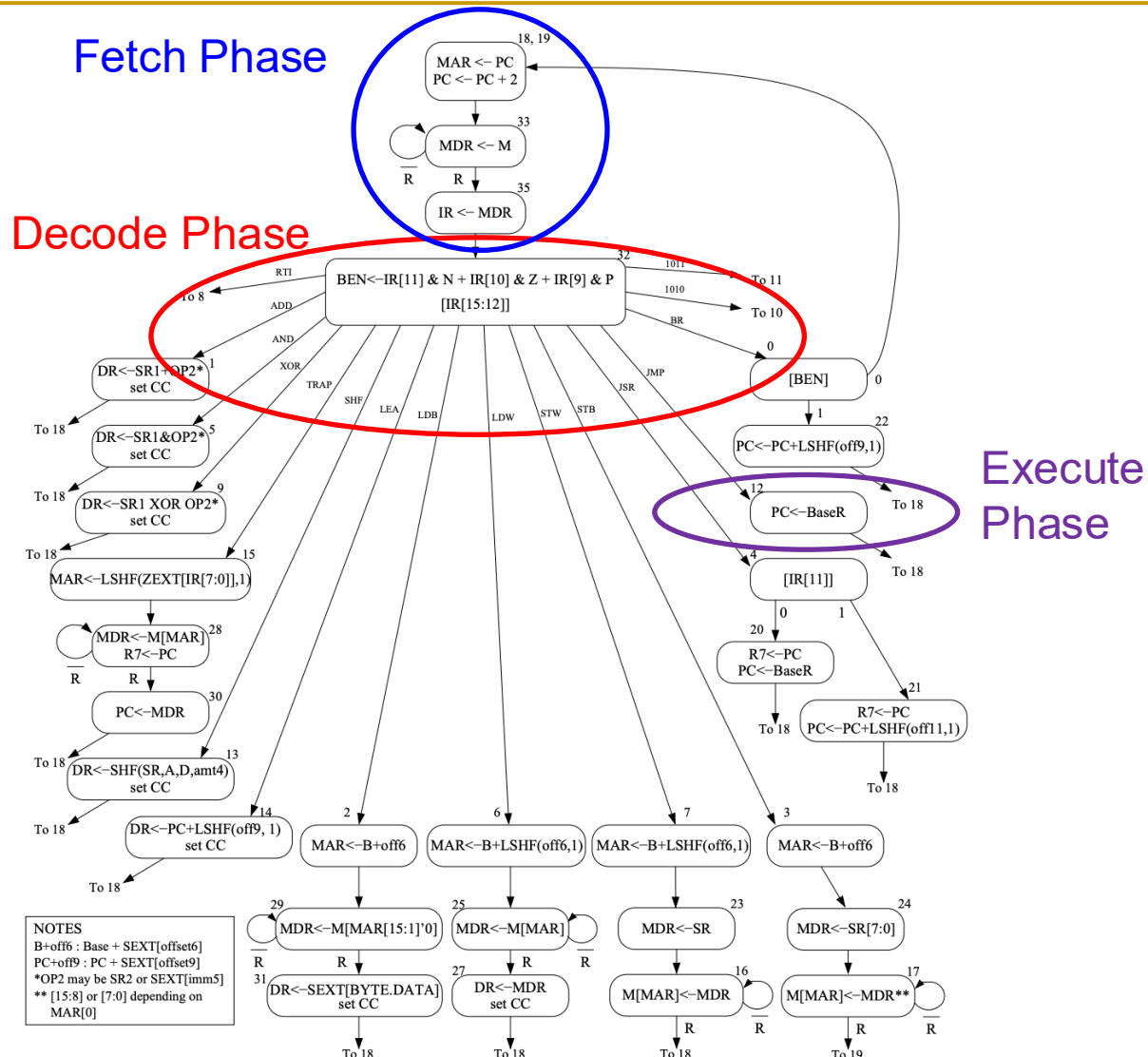


Figure C.2: A state machine for the LC-3b

Recall: LC-3: A von Neumann Machine

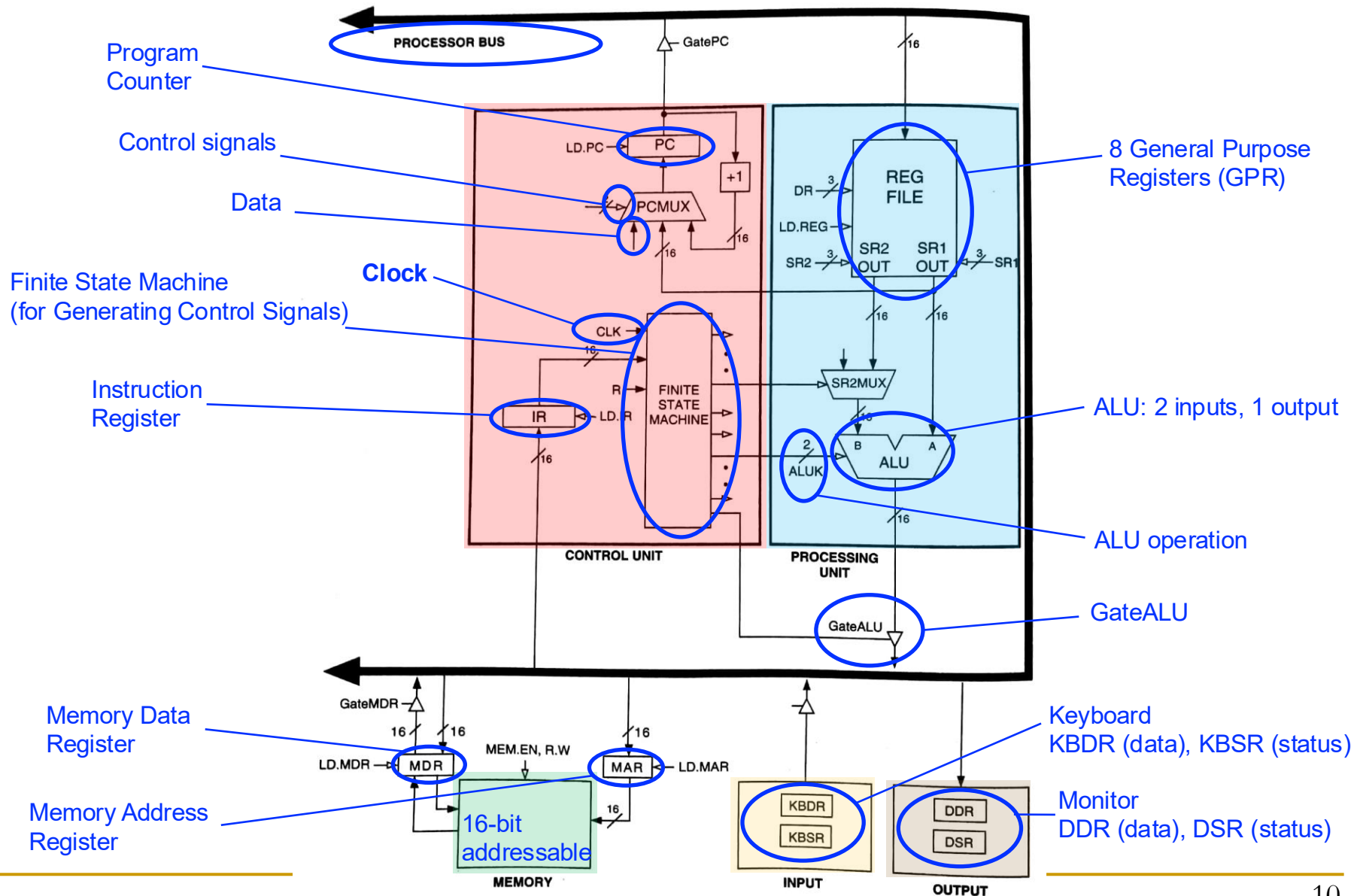


Figure 4.3 The LC-3 as an example of the von Neumann model

LC-3 and MIPS

Instruction Set Architectures

Instructions (Opcodes)

Data Types

Addressing Modes

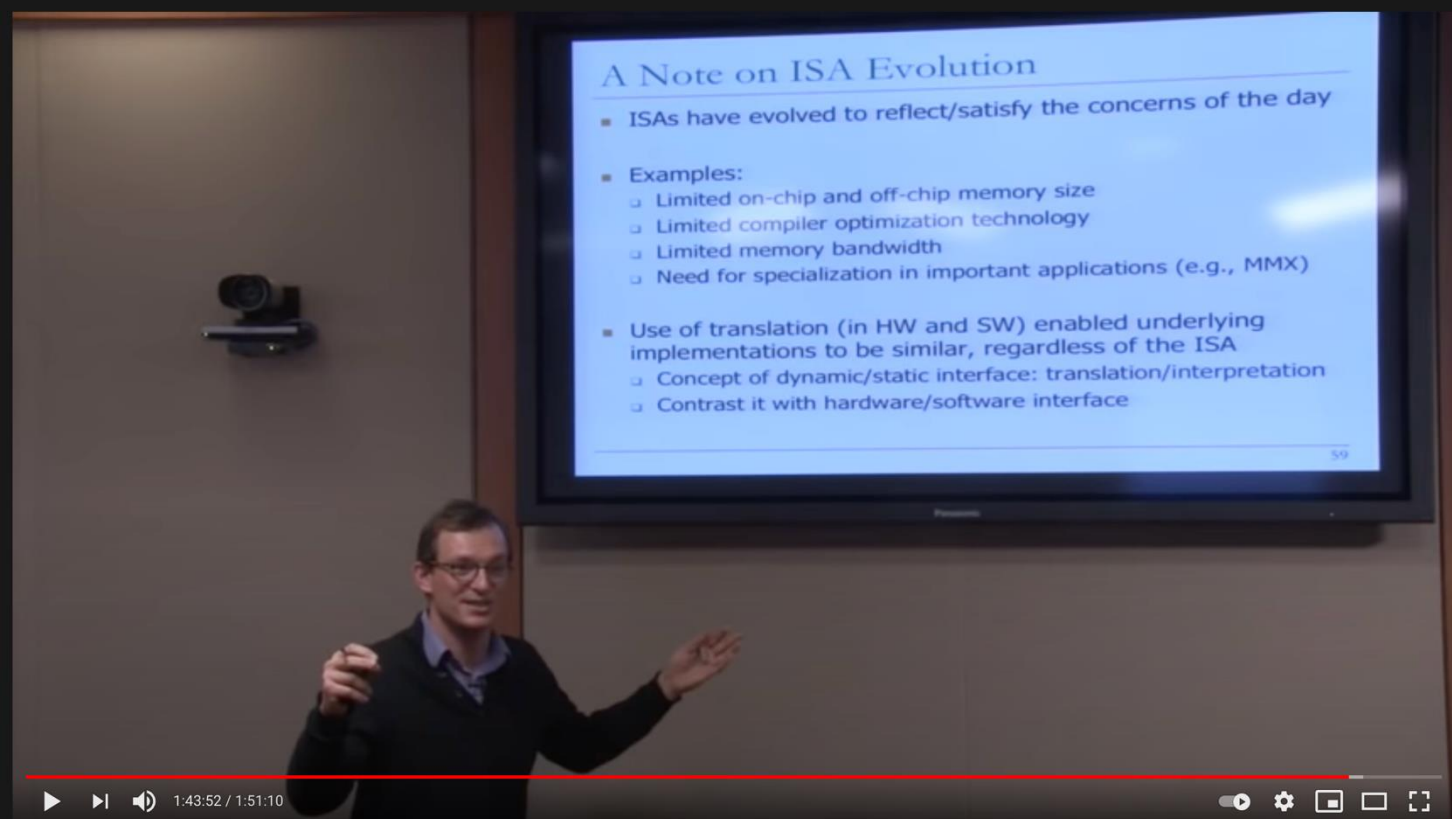
Operate Instructions

Data Movement Instructions and Addressing Modes

Control Flow Instructions

Conditional Control Flow (Conditional Branching)

There Is A Lot More to Cover on ISAs



A Note on ISA Evolution

- ISAs have evolved to reflect/satisfy the concerns of the day
- Examples:
 - Limited on-chip and off-chip memory size
 - Limited compiler optimization technology
 - Limited memory bandwidth
 - Need for specialization in important applications (e.g., MMX)
- Use of translation (in HW and SW) enabled underlying implementations to be similar, regardless of the ISA
 - Concept of dynamic/static interface: translation/interpretation
 - Contrast it with hardware/software interface

59

1:43:52 / 1:51:10

Lecture 3. ISA Tradeoffs - Carnegie Mellon - Computer Architecture 2015 - Onur Mutlu

44,973 views • Jan 24, 2015

Carnegie Mellon Computer Architecture
22.8K subscribers

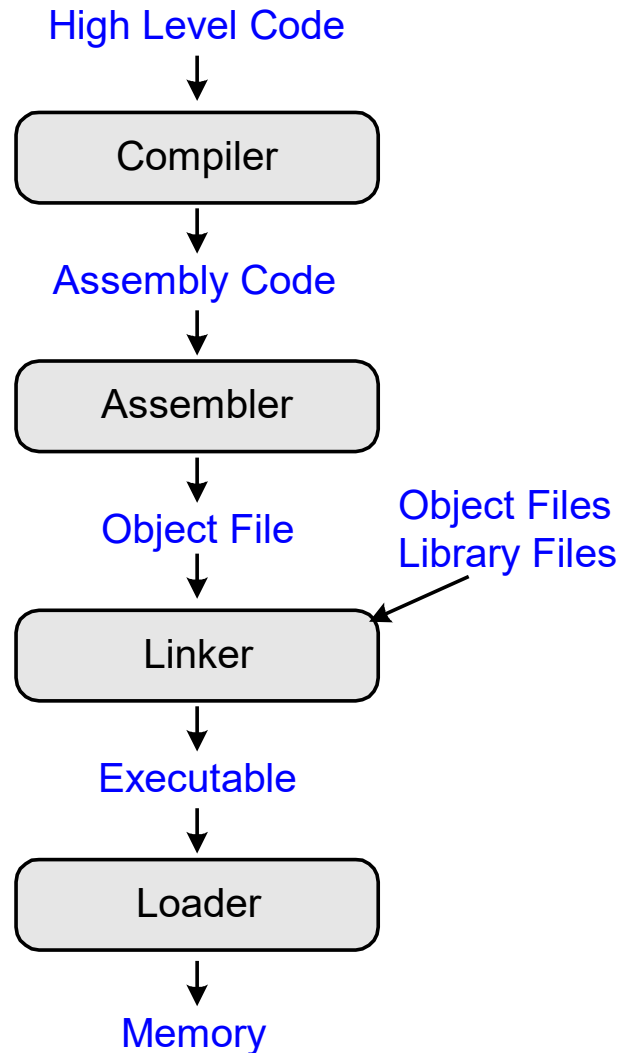
Lecture 3. ISA Tradeoffs
Lecturer: Prof. Onur Mutlu (<http://users.ece.cmu.edu/~omutlu/>)
Date: Jan 16th, 2015

276 5 SHARE SAVE ...

ANALYTICS EDIT VIDEO

Bigger Picture: Program Execution

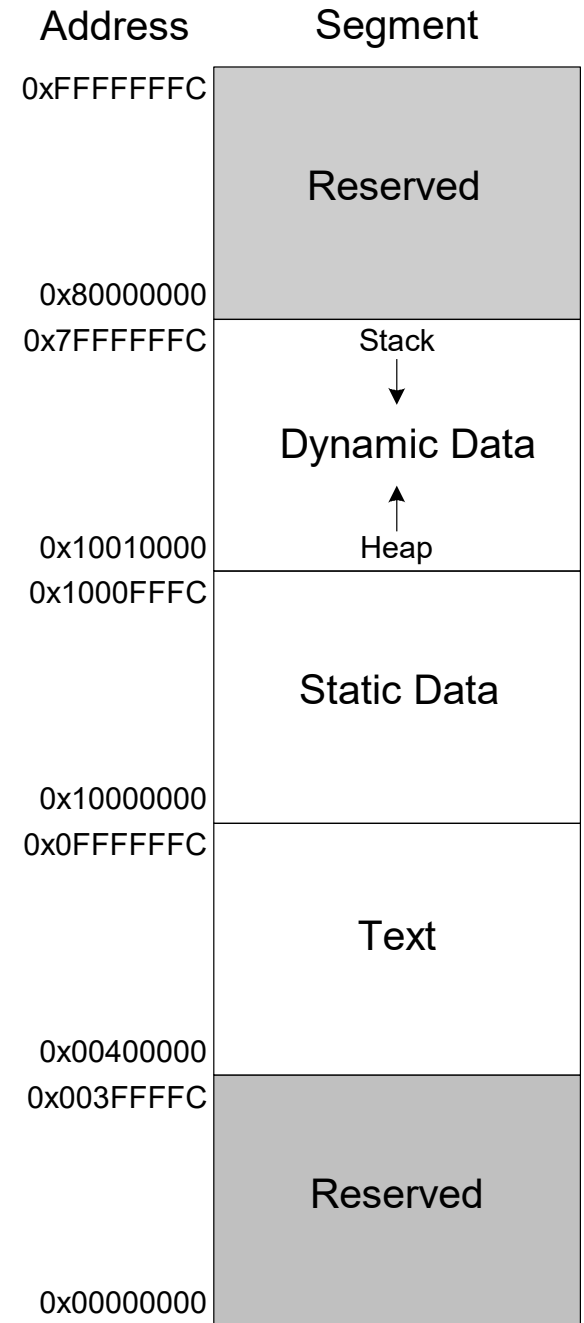
How Do We Compile & Run an Application?



What is Stored in Memory?

- **Instructions (also called text)**
- **Data**
 - Global/static: allocated before program begins
 - Dynamic: allocated within program
- **How big is memory (in MIPS ISA)?**
 - At most $2^{32} = 4$ gigabytes (4 GB)
 - From address `0x00000000` to `0xFFFFFFFF`

The MIPS Memory Map



Example Program: C Code

```
int f, g, y; // global variables

int main(void)
{
    f = 2;
    g = 3;
    y = sum(f, g);

    return y;
}

int sum(int a, int b) {
    return (a + b);
}
```


Example Program: Assembly Code

```
int f, g, y; // global
```

```
int main(void)
{

    f = 2;

    g = 3;

    y = sum(f, g);
    return y;
}
```

```
int sum(int a, int b) {
    return (a + b);
}
```

```
.data
```

```
f:
```

```
g:
```

```
y:
```

```
.text
```

```
main: addi $sp, $sp, -4 # stack
      sw   $ra, 0($sp)  # store $ra
      addi $a0, $0, 2   # $a0 = 2
      sw   $a0, f       # f = 2
      addi $a1, $0, 3   # $a1 = 3
      sw   $a1, g       # g = 3
      jal  sum          # call sum
      sw   $v0, y       # y = sum()
      lw   $ra, 0($sp)  # rest. $ra
      addi $sp, $sp, 4  # rest. $sp
      jr   $ra          # return
sum:   add  $v0, $a0, $a1 # $v0= a+b
      jr   $ra          # return
```

Example Program: Symbol Table

Symbol	Address
f	0x10000000
g	0x10000004
y	0x10000008
main	0x00400000
sum	0x0040002C

Example Program: Executable

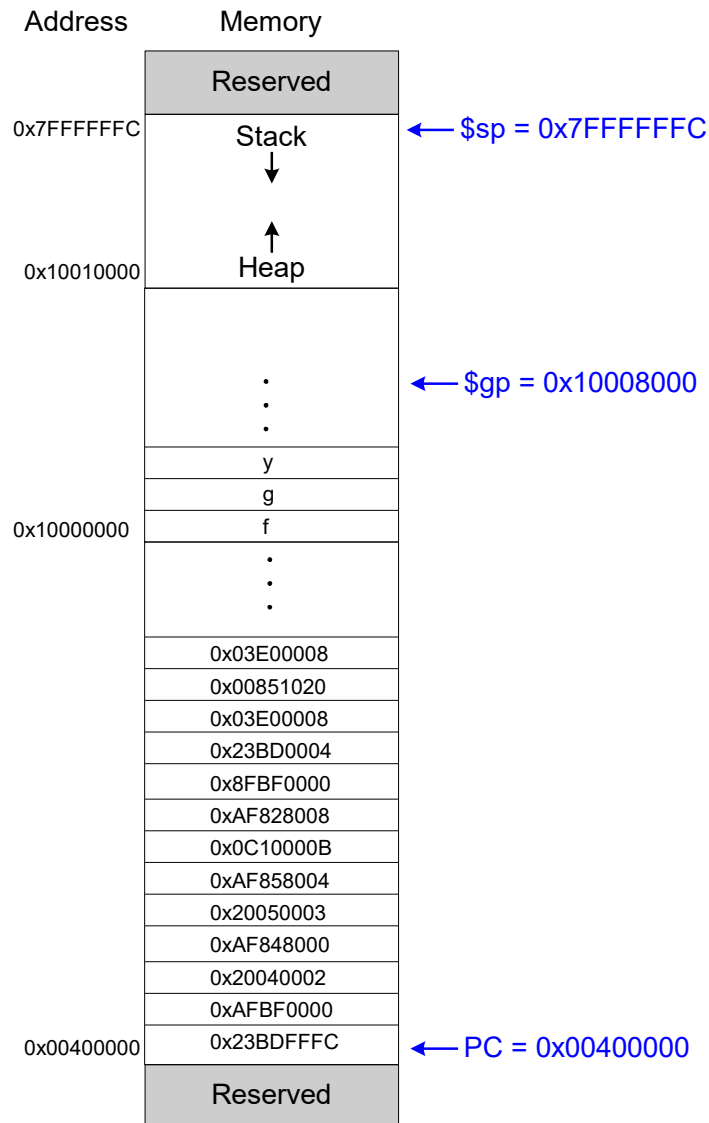
Executable file header	Text Size	Data Size
	0x34 (52 bytes)	0xC (12 bytes)
Text segment	Address	Instruction
	0x00400000	0x23BDFFFC
	0x00400004	0xAFBF0000
	0x00400008	0x20040002
	0x0040000C	0xAF848000
	0x00400010	0x20050003
	0x00400014	0xAF858004
	0x00400018	0x0C10000B
	0x0040001C	0xAF828008
	0x00400020	0x8FBF0000
	0x00400024	0x23BD0004
	0x00400028	0x03E00008
	0x0040002C	0x00851020
	0x00400030	0x03E00008
Data segment	Address	Data
	0x10000000	f
	0x10000004	g
	0x10000008	y

```

addi $sp, $sp, -4
sw   $ra, 0($sp)
addi $a0, $0, 2
sw   $a0, 0x8000($gp)
addi $a1, $0, 3
sw   $a1, 0x8004($gp)
jal  0x0040002C
sw   $v0, 0x8008($gp)
lw   $ra, 0($sp)
addi $sp, $sp, 4
jr   $ra
add  $v0, $a0, $a1
jr   $ra

```

Example Program: In Memory

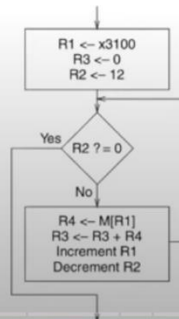


Extra Lecture on Assembly Programming

A Program for Adding Integers in LC-3

- We use conditional branch instructions to create a loop

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	LEA	1	1	0	0	0	1	0	x00FF	1	1	1	1	1	1	1	R1 = PC + 0x00FF = 3100 // load address
x3001	AND	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 = 0 // reset register
x3002	AND	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 = 0 // reset register
x3003	ADD	0	0	1	0	1	0	0	1	0	1	0	1	1	0	0	R2 = R2 + 12 // initialize counter
x3004	BR	0	0	0	0	z	0	5	0	0	0	0	0	0	1	0	BRz (PC + 5) = BRz 0x300A // check condition
x3005	LDR	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	R4 = M[R1 + 0] // load value
x3006	ADD	0	0	1	0	1	1	0	1	1	0	0	0	1	0	0	R3 = R3 + R4 // accumulate
x3007	ADD	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	R1 = R1 + 1 // increment address
x3008	ADD	0	0	1	0	1	0	0	1	0	1	1	1	1	1	1	R2 = R2 - 1 // decrement counter
x3009	BR	0	0	0	n	z	p	6	1	1	1	1	1	0	1	0	BRnzp (PC + 6) = BRnzp 0x3004 // jump



Digital Design & Computer Architecture - Lecture 10b: Assembly Programming (Spring 2022)



Onur Mutlu Lectures
31.8K subscribers

Analytics

Edit video

46



Share

Download

Clip

Save



2K views 11 months ago Livestream - Digital Design and Computer Architecture - ETH Zürich (Spring 2022)

Digital Design and Computer Architecture, ETH Zürich, Spring 2022 (<https://safari.ethz.ch/digitaltechnik...>)

Extra Lecture on Assembly Programming

Function Calls: Code Example

High-level code

```
int main()
{
    int y;
    ...
    // 4 arguments
    y = diffofsums(2, 3, 4, 5);
    ...
}

int diffofsums(int f, int g,
               int h, int i)
{
    int result;
    result = (f + g) - (h + i);
    // return value
    return result;
}
```

MIPS assembly

```
# $s0 = y
main:
...
addi $a0, $0, 2    # argument 0 = 2
addi $a1, $0, 3    # argument 1 = 3
addi $a2, $0, 4    # argument 2 = 4
addi $a3, $0, 5    # argument 3 = 5
jal  diffofsums    # call procedure
add  $s0, $v0, $0   # y = returned value
...

# $s0 = result
diffofsums:
add  $t0, $a0, $a1  # $t0 = f + g
add  $t1, $a2, $a3  # $t1 = h + i
sub  $s0, $t0, $t1  # result=(f + g) - (h + i)
add  $v0, $s0, $0   # put return value in $v0
jr   $ra            # return to caller
```

Argument values (red arrow pointing to \$a0-\$a3)

Return value (blue arrow pointing to \$v0)

Return address (green arrow pointing to \$ra)

Digital Design & Computer Architecture - Lecture 10b: Assembly Programming (Spring 2022)



Onur Mutlu Lectures
31.8K subscribers

Analytics

Edit video

46



Share

Download

Clip

Save



2,069 views Premiered Mar 28, 2022 Livestream - Digital Design and Computer Architecture - ETH Zürich (Spring 2022)
Digital Design and Computer Architecture, ETH Zürich, Spring 2022 (<https://safari.ethz.ch/digitaltechnik...>)

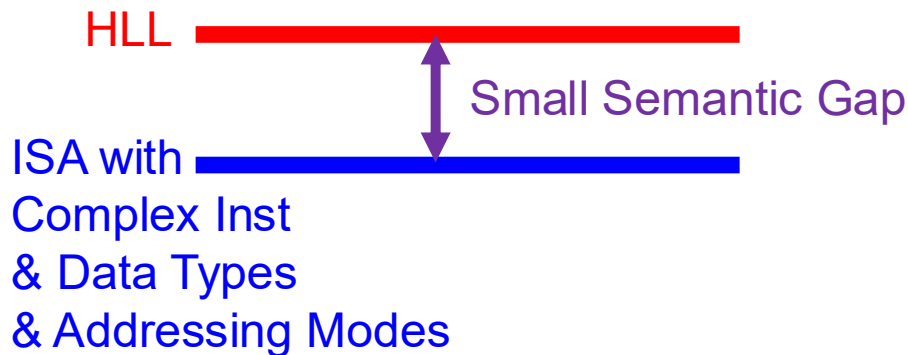
Some More on ISA Tradeoffs

Complex vs. Simple Instructions+Data Types

- **Complex instruction**: An instruction **does a lot of work**, e.g. many operations
 - ❑ Insert in a doubly linked list
 - ❑ Compute FFT
 - ❑ String copy
 - ❑ Matrix multiply
 - ❑ ...
- **Simple instruction**: An instruction **does little work** -- it is a primitive using which complex operations can be built
 - ❑ Add
 - ❑ XOR
 - ❑ Multiply
 - ❑ ...

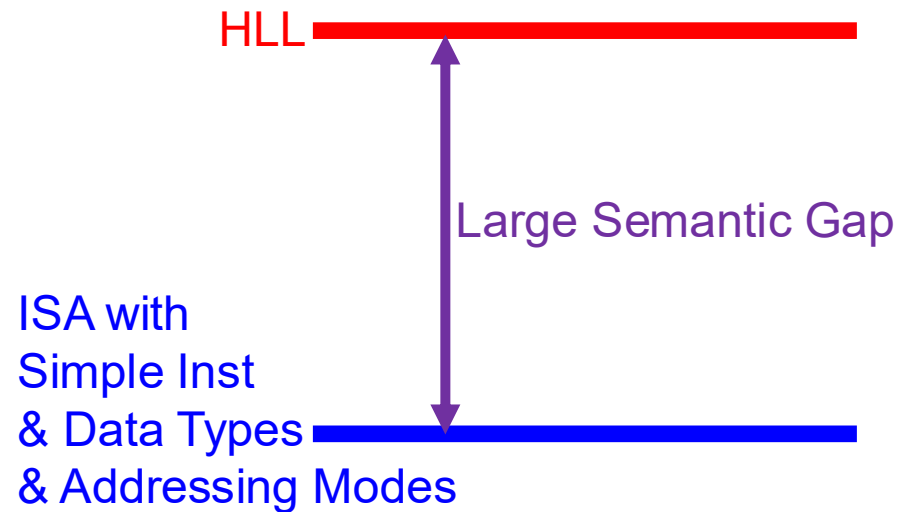
Recall: Semantic Gap

- How close instructions & data types & addressing modes are to high-level language (HLL)



HW Control Signals

Easier mapping of HLL to ISA
Less work for software designer
More work for hardware designer
Optimization burden on HW

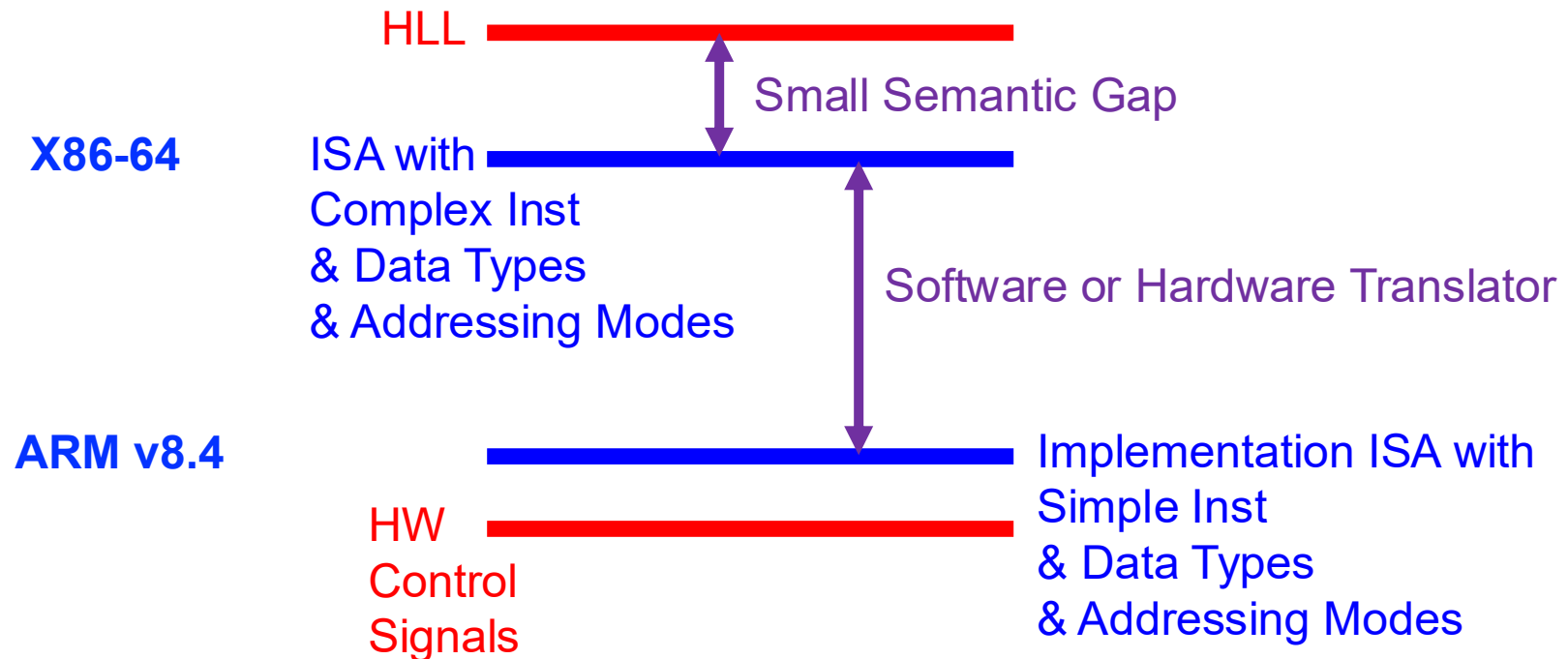


HW Control Signals

Harder mapping of HLL to ISA
More work for software designer
Less work for hardware designer
Optimization burden on SW

How to Change the Semantic Gap Tradeoffs

- Translate from one ISA into a different “implementation” ISA



An Example: Rosetta 2 Binary Translator

Rosetta 2 [\[edit \]](#)

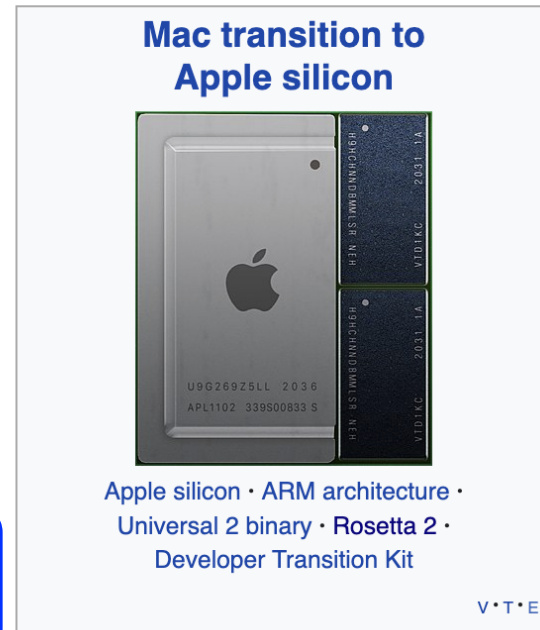
In 2020, Apple announced Rosetta 2 would be bundled with [macOS Big Sur](#), to aid in the [Mac transition to Apple silicon](#). The software permits many applications compiled exclusively for execution on x86-64-based processors to be translated for execution on Apple silicon.^{[2][8]}

In addition to the [just-in-time](#) (JIT) translation support, Rosetta 2 offers [ahead-of-time compilation](#) (AOT), with the x86-64 code fully translated, just once, when an application without a universal binary is installed on an Apple silicon Mac.^[9]

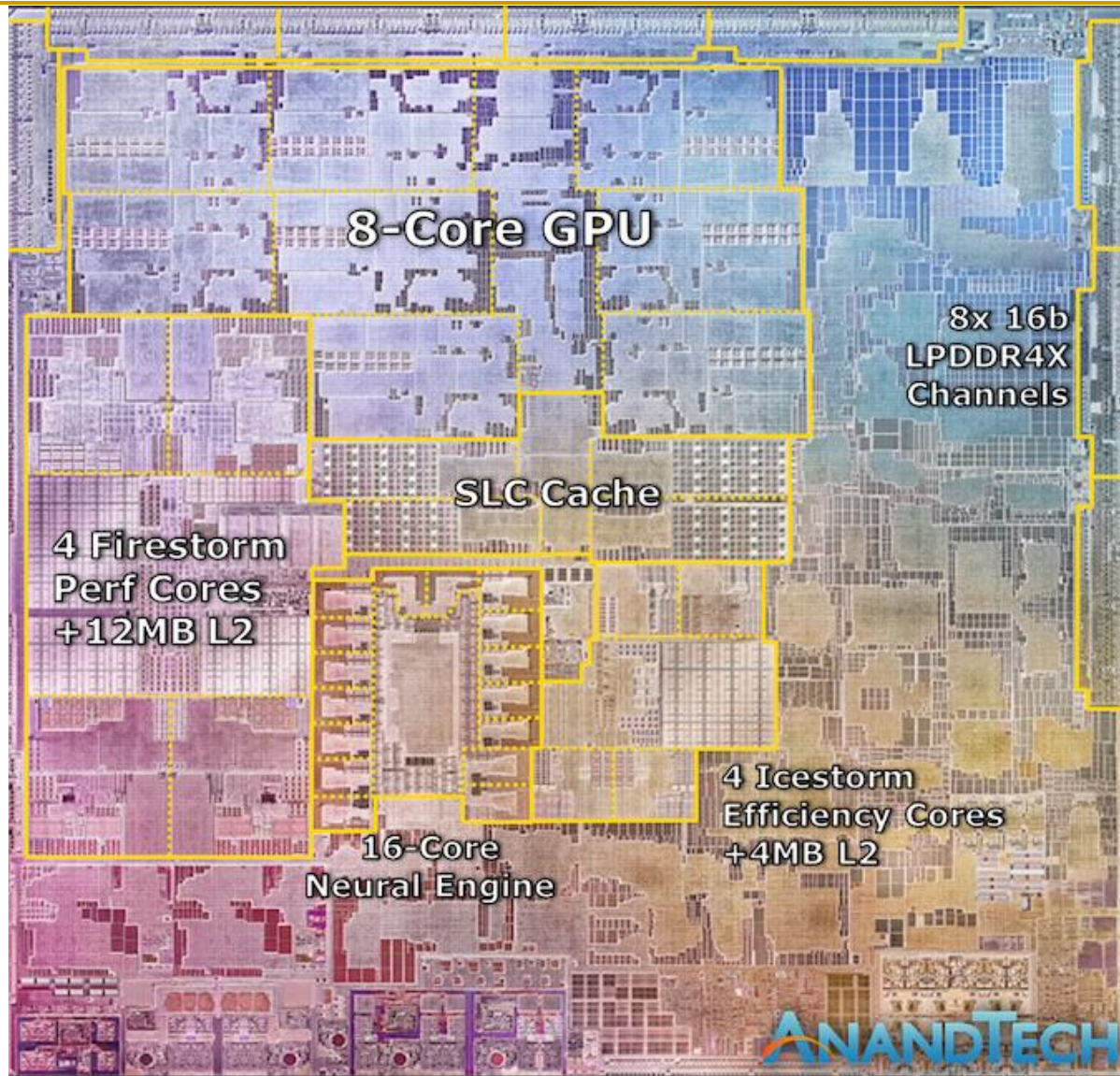
Rosetta 2's performance has been praised greatly.^{[10][11]} In some benchmarks, x86-64-only programs performed better under Rosetta 2 on a Mac with an Apple M1 SOC than natively on a Mac with an Intel x86-64 processor. One of the key reasons why Rosetta 2 provides such high level of translation efficiency is the support of x86-64 [memory ordering](#) in Apple M1 SOC.^[12]

Although Rosetta 2 works for most software, some software doesn't work at all^[13] or is reported to be "sluggish".^[14] A lot of software can be made compatible with the new Macs by the vendor recompiling the software, often a simple task; while for some software (such as software that includes [assembly language](#) code, or that generates [machine code](#)), the changes to make them work aren't simple and cannot be automated.

Similar to the first version, Rosetta 2 does not normally require user intervention. When a user attempts to launch an x86-64-only application for the first time, macOS prompts them to install Rosetta 2 if it is not already available. Subsequent launches of x86-64 programs will execute via translation automatically. An option also exists to force a universal binary to run as x86-64 code through Rosetta 2, even on an ARM-based machine.^[15]

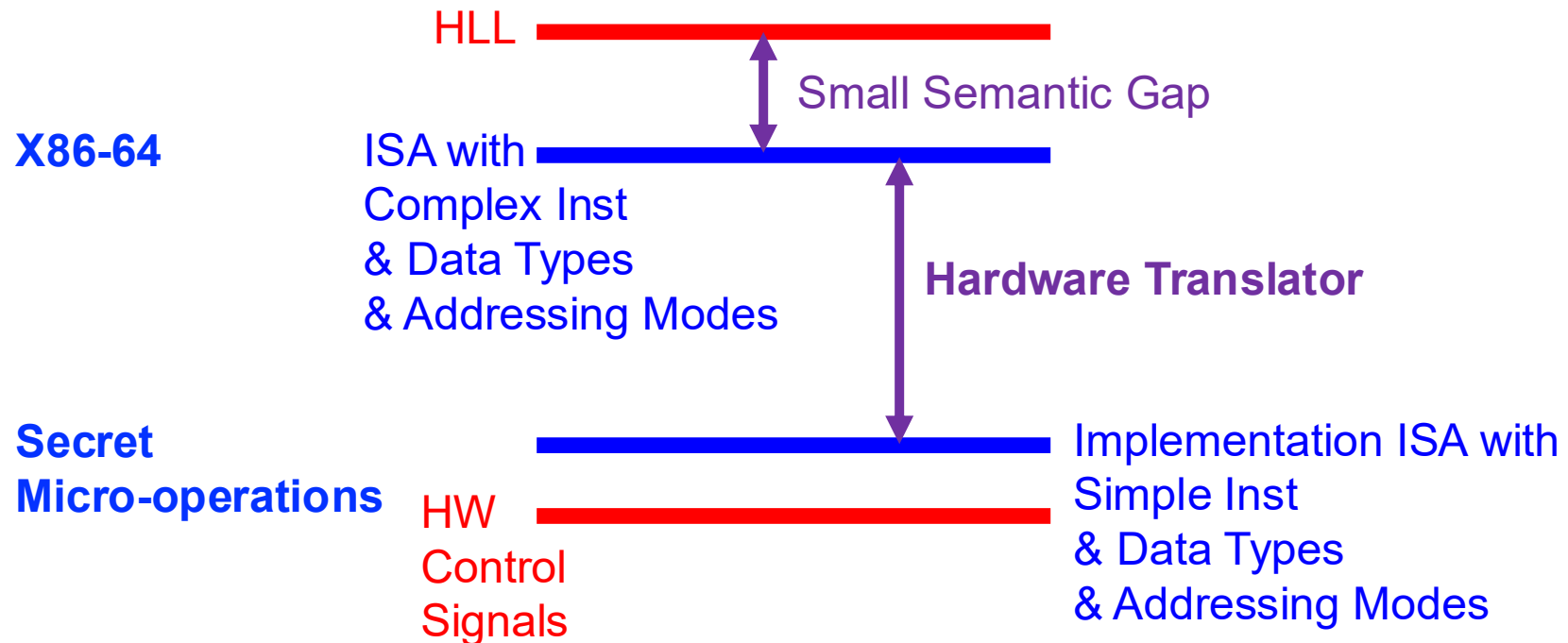


An Example: Rosetta 2 Binary Translator



Apple M1,
2021

Another Example: Intel and AMD Processors



Another Example: Intel and AMD Processors

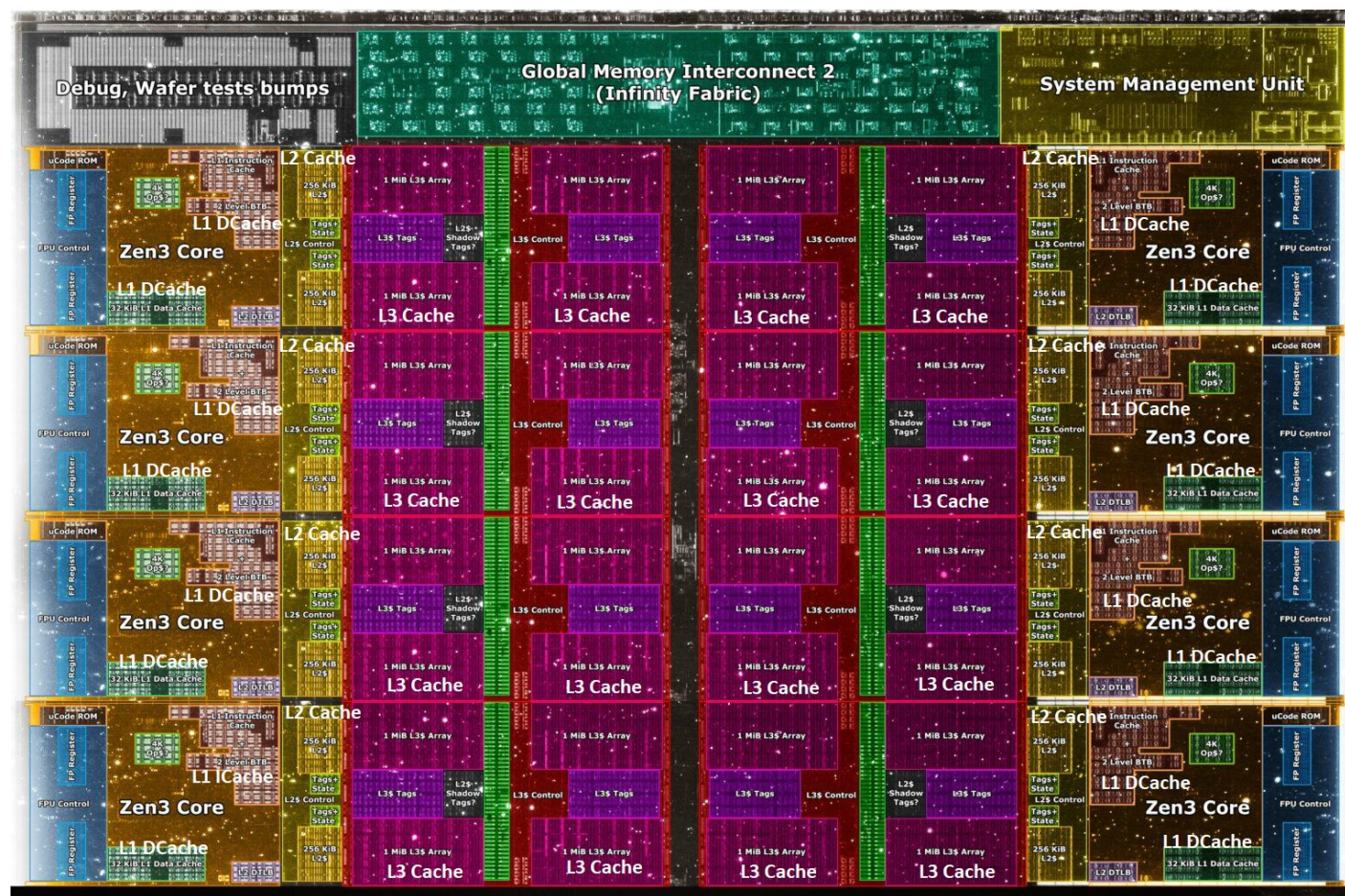


10nm ESF=Intel 7 Alder Lake die shot (~209mm²) from Intel: <https://www.intel.com/content/www/us/en/newsroom/news/12th-gen-core-processors.html>

Die shot interpretation by Locuza, October 2021

Intel Alder Lake,
2021

Another Example: Intel and AMD Processors



Core Count:
8 cores/16 threads

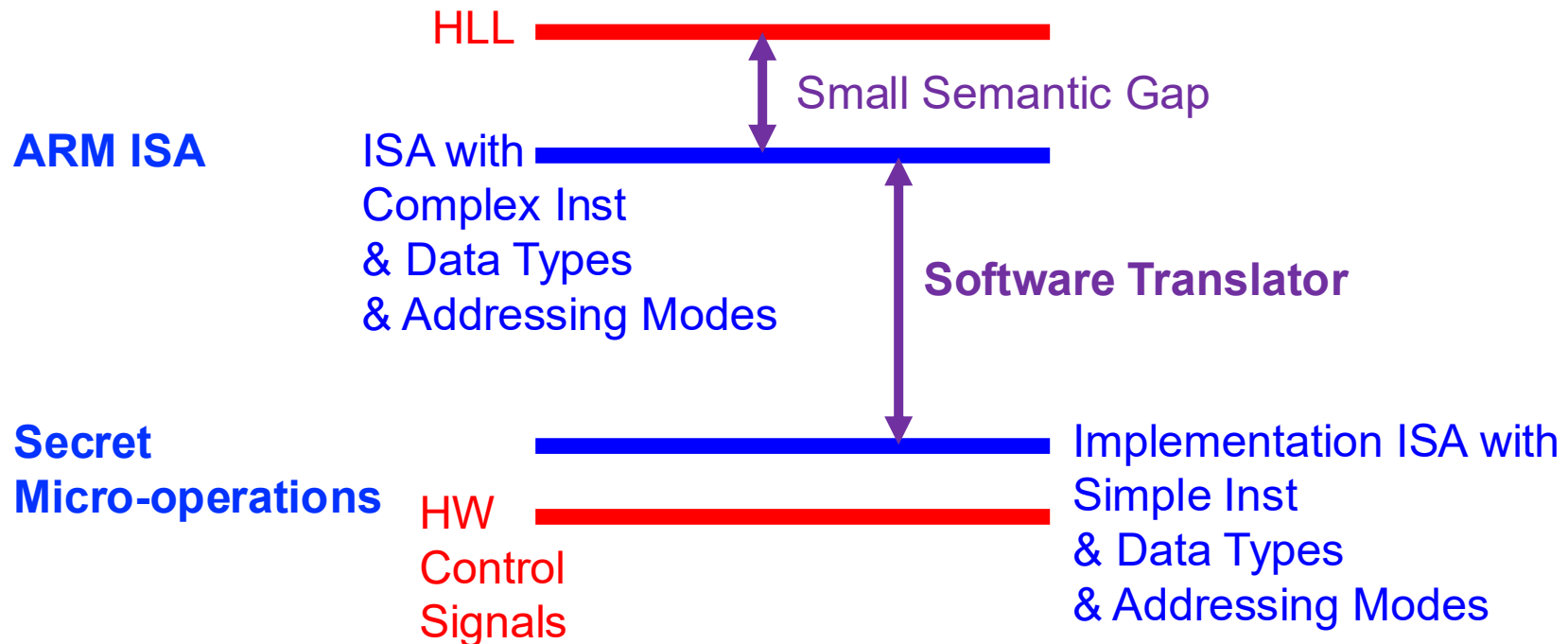
L1 Caches:
32 KB per core

L2 Caches:
512 KB per core

L3 Cache:
32 MB shared

AMD Ryzen 5000, 2020

Another Example: NVIDIA Denver



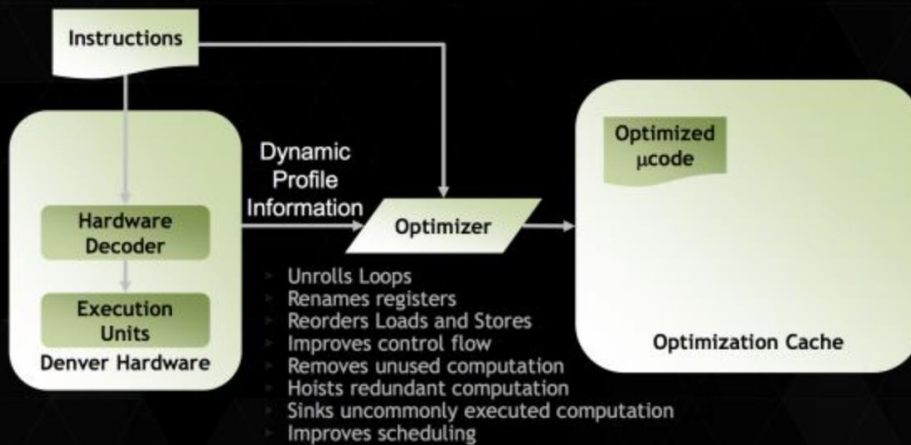
Another Example: NVIDIA Denver

The Secret of Denver: Binary Translation & Code Optimization

As we alluded to earlier, NVIDIA's decision to forgo a traditional out-of-order design for Denver means that much of Denver's potential is contained in its software rather than its hardware. The underlying chip itself, though by no means simple, is at its core a very large in-order processor. So it falls to the software stack to make Denver sing.

Accomplishing this task is NVIDIA's dynamic code optimizer (DCO). The purpose of the DCO is to accomplish two tasks: to translate ARM code to Denver's native format, and to optimize this code to make it run better on Denver. With no out-of-order hardware on Denver, it is the DCO's task to find instruction level parallelism within a thread to fill Denver's many execution units, and to reorder instructions around potential stalls, something that is no simple task.

DYNAMIC CODE OPTIMIZATION OPTIMIZE ONCE, USE MANY TIMES



Transmeta: x86 to VLIW Translation

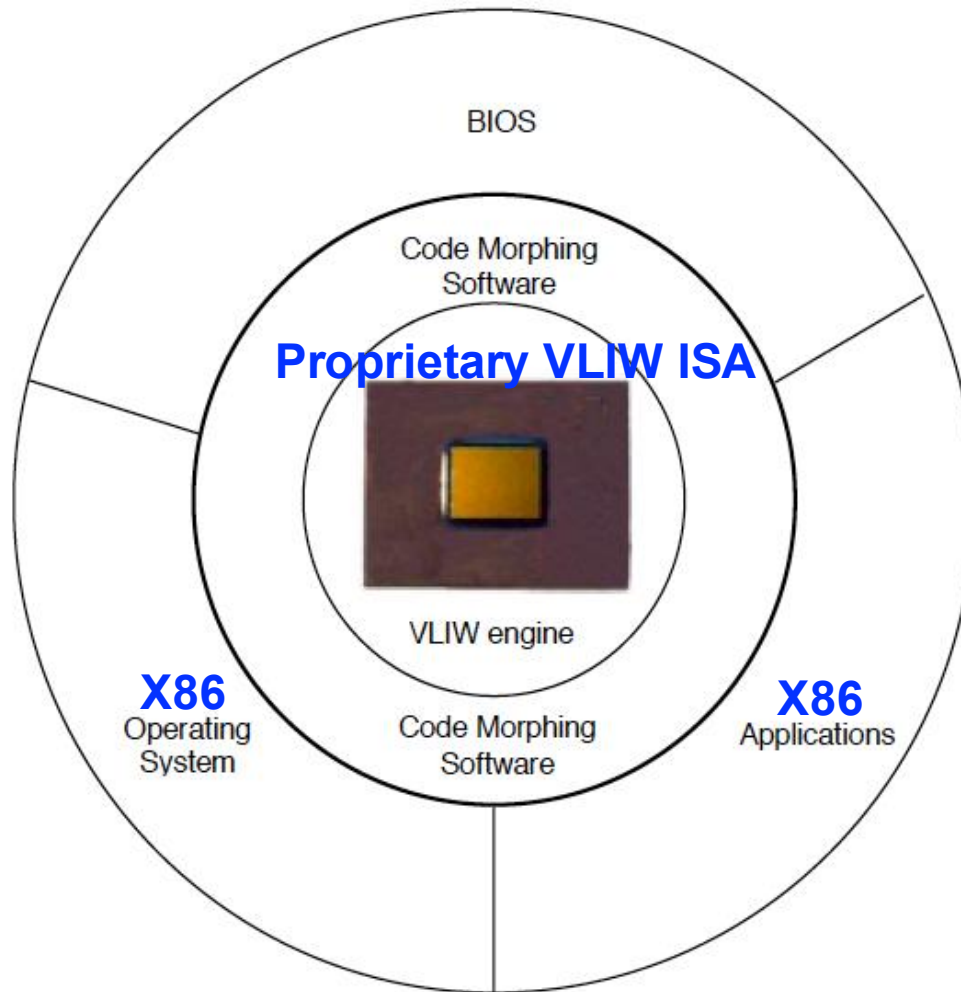


Figure 5. The Code Morphing software mediates between x86 software and the Crusoe processor.

Klaiber, “[The Technology Behind Crusoe Processors](#),” Transmeta White Paper 2000.

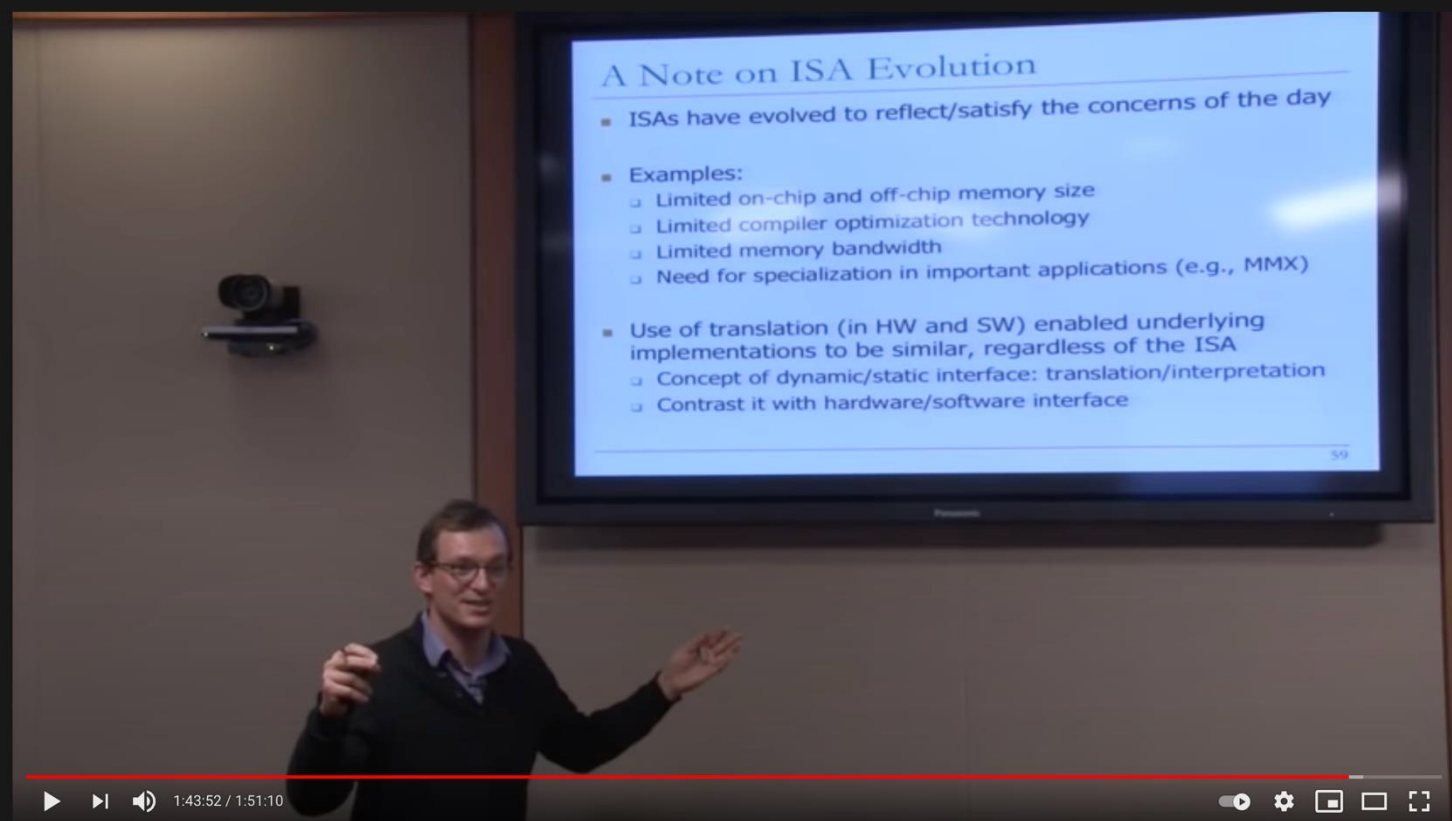
Principle: Indirection

- “Any problem in computer science can be solved with another level of indirection.”
 - David Wheeler
 - Attributed in Butler Lampson’s “Principles for Computer Systems Design” <https://arxiv.org/pdf/2011.02455>
- Tradeoffs can change and new opportunities become enabled once you add a level of indirection
- But indirection comes with extra complexity and latency
- We will see this again when we discuss **Virtual Memory**

ISA-level Tradeoffs: Number of Registers

- Affects:
 - Number of bits used for encoding register address
 - Number of values kept in fast storage (register file)
 - (uarch) Size, access time, power consumption of register file
- Large number of registers:
 - + Enables better register allocation (and optimizations) by compiler → fewer saves/restores
 - Larger instruction size
 - Larger register file size
- We already saw this tradeoff: LC-3 vs MIPS

There Is A Lot More to Cover on ISAs



A Note on ISA Evolution

- ISAs have evolved to reflect/satisfy the concerns of the day
- Examples:
 - Limited on-chip and off-chip memory size
 - Limited compiler optimization technology
 - Limited memory bandwidth
 - Need for specialization in important applications (e.g., MMX)
- Use of translation (in HW and SW) enabled underlying implementations to be similar, regardless of the ISA
 - Concept of dynamic/static interface: translation/interpretation
 - Contrast it with hardware/software interface

59

1:43:52 / 1:51:10

Lecture 3. ISA Tradeoffs - Carnegie Mellon - Computer Architecture 2015 - Onur Mutlu

44,973 views • Jan 24, 2015

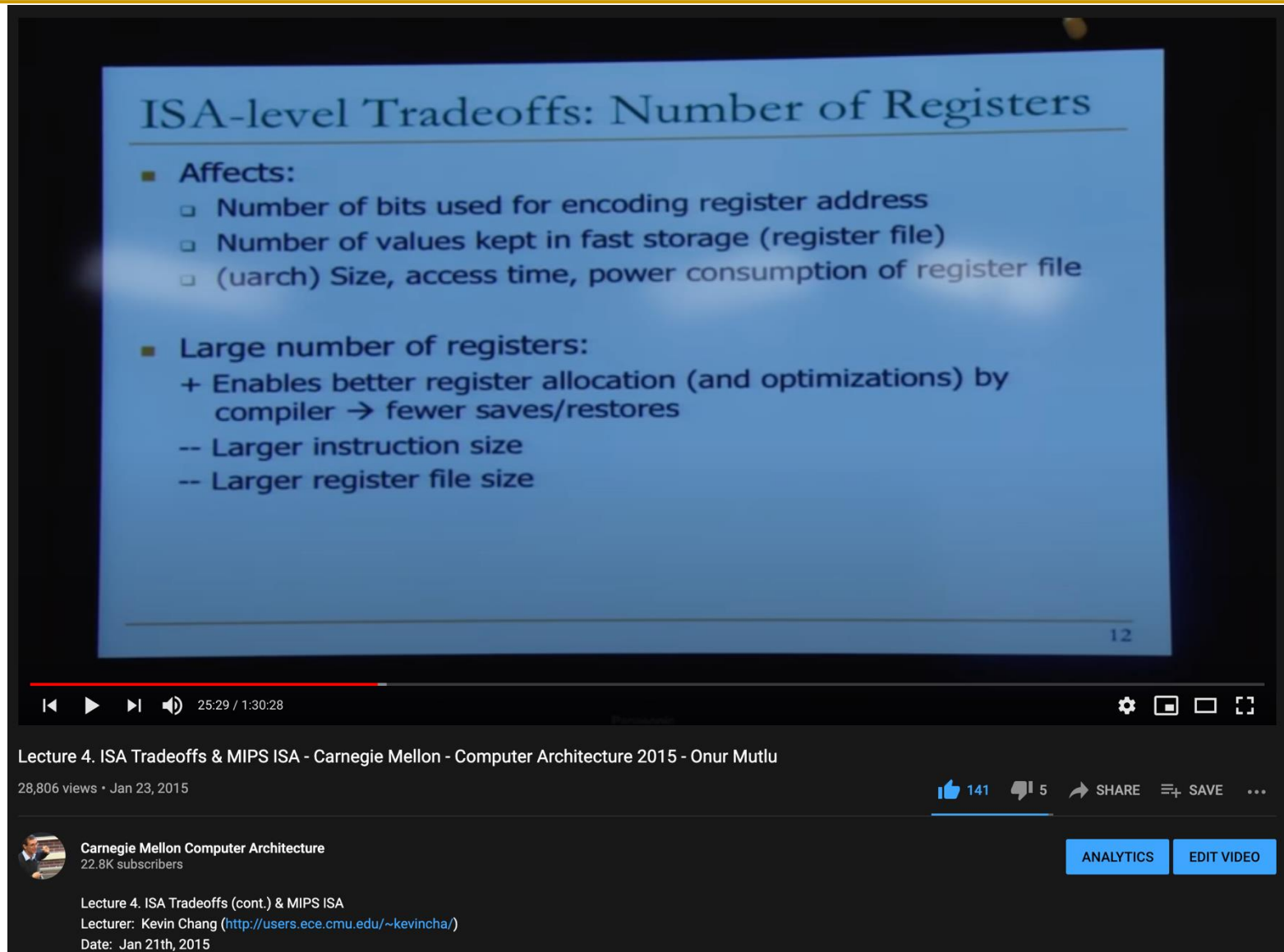
Carnegie Mellon Computer Architecture
22.8K subscribers

Lecture 3. ISA Tradeoffs
Lecturer: Prof. Onur Mutlu (<http://users.ece.cmu.edu/~omutlu/>)
Date: Jan 16th, 2015

276 5 SHARE SAVE ...

ANALYTICS EDIT VIDEO

There Is A Lot More to Cover on ISAs



The video player displays a slide with the following content:

ISA-level Tradeoffs: Number of Registers

- **Affects:**
 - Number of bits used for encoding register address
 - Number of values kept in fast storage (register file)
 - (uarch) Size, access time, power consumption of register file
- **Large number of registers:**
 - + Enables better register allocation (and optimizations) by compiler → fewer saves/restores
 - Larger instruction size
 - Larger register file size

12

Lecture 4. ISA Tradeoffs & MIPS ISA - Carnegie Mellon - Computer Architecture 2015 - Onur Mutlu

28,806 views • Jan 23, 2015

141 5 SHARE SAVE ...

 **Carnegie Mellon Computer Architecture**
22.8K subscribers

Lecture 4. ISA Tradeoffs (cont.) & MIPS ISA
Lecturer: Kevin Chang (<http://users.ece.cmu.edu/~kevincha/>)
Date: Jan 21th, 2015

ANALYTICS EDIT VIDEO

Detailed Lectures on ISAs & ISA Tradeoffs

■ Computer Architecture, Spring 2015, Lecture 3

- ISA Tradeoffs (CMU, Spring 2015)
- <https://www.youtube.com/watch?v=QKdiZSfwg-g&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=3>

■ Computer Architecture, Spring 2015, Lecture 4

- ISA Tradeoffs & MIPS ISA (CMU, Spring 2015)
- <https://www.youtube.com/watch?v=RBgeCCW5Hjs&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=4>

■ Computer Architecture, Spring 2015, Lecture 2

- Fundamental Concepts and ISA (CMU, Spring 2015)
- <https://www.youtube.com/watch?v=NpC39uS4K4o&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=2>

ISA Design and Tradeoffs: More Critical Thinking

The Von Neumann Model/Architecture

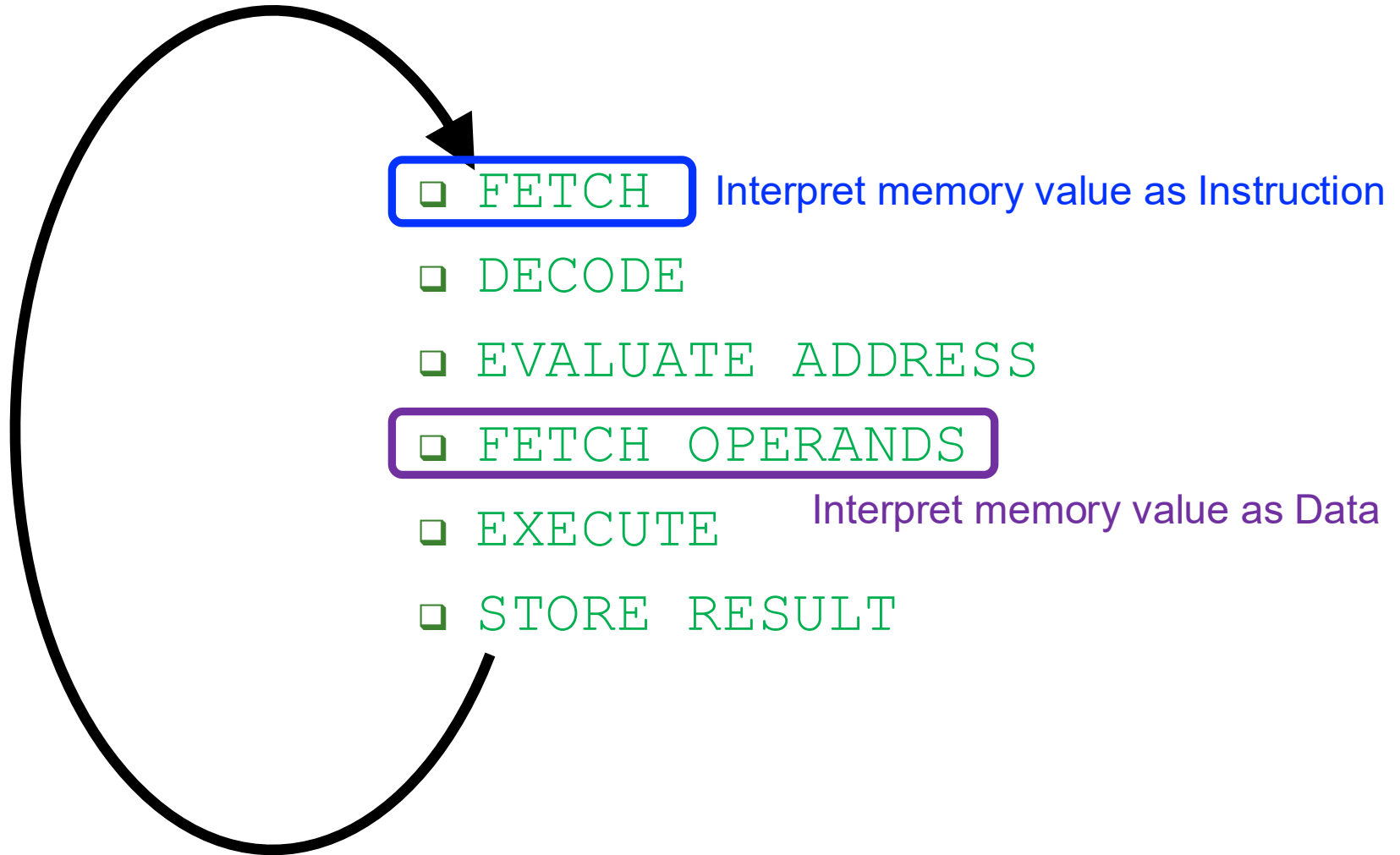
Stored program

Sequential instruction processing

The von Neumann Model/Architecture

- Von Neumann model is also called *stored program computer* (instructions in memory). It has two key properties:
- **Stored program**
 - Instructions stored in a linear memory array
 - **Memory is unified** between instructions and data
 - The interpretation of a stored value depends on the control signals
When is a value interpreted as an instruction?
- **Sequential instruction processing**

Recall: The Instruction Cycle



Whether a value fetched from memory is interpreted as an instruction depends on **when** that value is **fetched** in the instruction processing cycle.

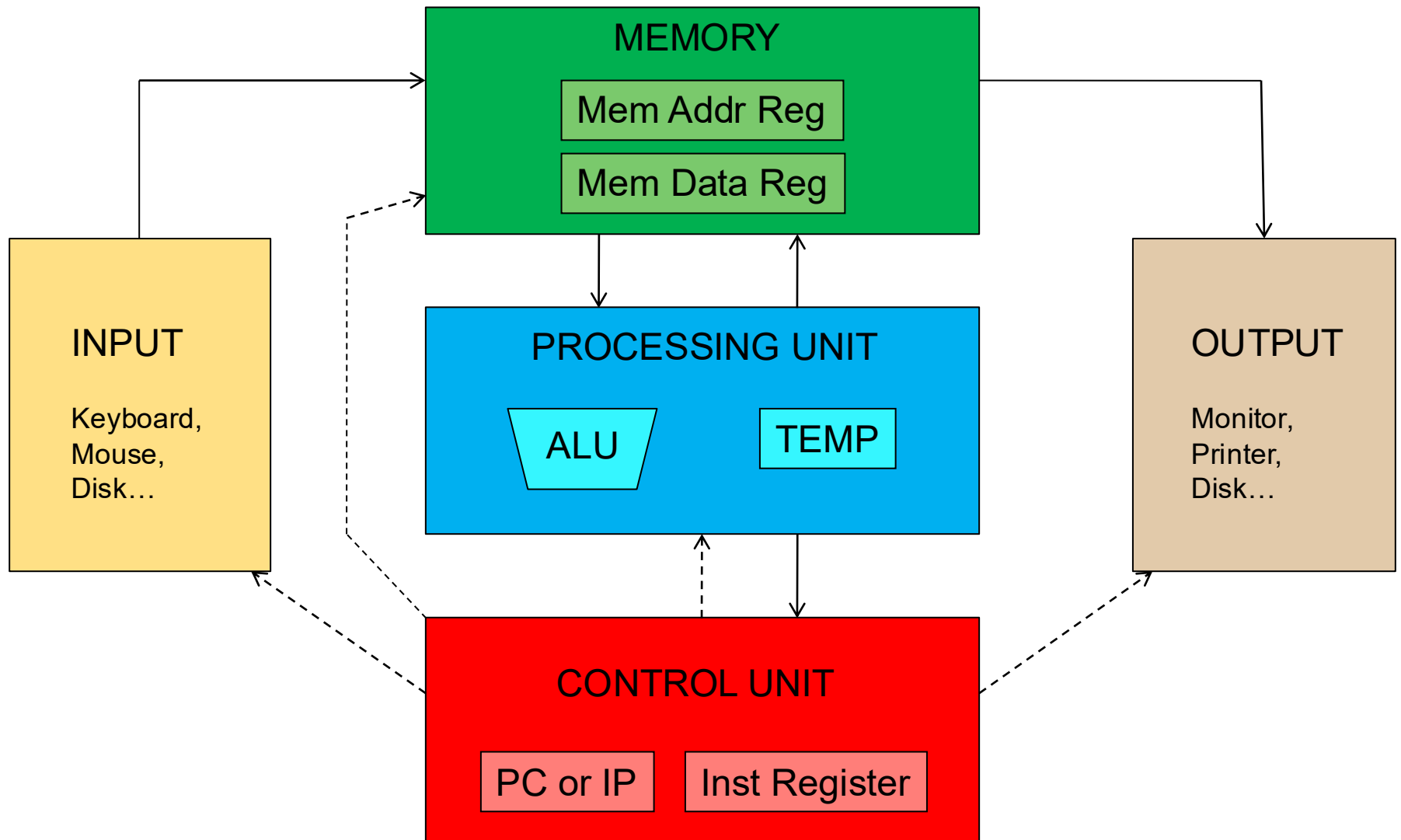
The von Neumann Model/Architecture

- Von Neumann model is also called *stored program computer* (instructions in memory). It has two key properties:
- **Stored program**
 - Instructions stored in a linear memory array
 - **Memory is unified** between instructions and data
 - **The interpretation of a stored value depends on the control signals**
When is a value interpreted as an instruction?
- **Sequential instruction processing**
 - One instruction processed (fetched, executed, completed) at a time
 - **Program counter (instruction pointer)** identifies the current instruction
 - **Program counter is advanced sequentially** except for control transfer instructions

The von Neumann Model/Architecture

- Recommended reading
 - Burks, Goldstein, von Neumann, "Preliminary discussion of the logical design of an electronic computing instrument," 1946.
- Important reading
 - Patt and Patel book, Chapter 4, "The von Neumann Model"
- **Stored program**
- **Sequential instruction processing**

The Von Neumann Model (of a Computer)



The Von Neumann Model (of a Computer)

- Q: Is this the only way that a computer can process computer programs?

The von Neumann Model

- In order to build a computer, we need an execution model for processing computer programs

- John von Neumann proposed a fundamental model in 1946

- The von Neumann Model consists of 5 components

- Memory (stores the program and data)
- Processing unit
- Input
- Output
- Control unit (controls the order in which instructions are carried out)



- Throughout this lecture, we will examine two examples of the von Neumann model

- LC-3
- MIPS

Burks, Goldstein, von Neumann,
"Preliminary discussion of the logical design
of an electronic computing instrument," 1946.

All general-purpose computers today use the von Neumann model

14

- A: No.
- Qualified Answer: No. But, it has been the dominant way
 - i.e., the dominant paradigm for computing
 - for N decades

The Dataflow Execution Model of a Computer

The Dataflow Model (of a Computer)

- **Von Neumann model:** An instruction is fetched and executed in **control flow order**
 - ❑ As specified by the **program counter (instruction pointer)**
 - ❑ Sequential unless explicit control flow instruction

- **Dataflow model:** An instruction is fetched and executed in **data flow order**
 - ❑ i.e., when its operands are ready
 - ❑ i.e., there is **no program counter (instruction pointer)**
 - ❑ Instruction ordering specified by data flow dependence
 - Each instruction specifies “who” should receive the result
 - An instruction can “fire” whenever all operands are received
 - ❑ Potentially many instructions can execute at the same time
 - Inherently more parallel

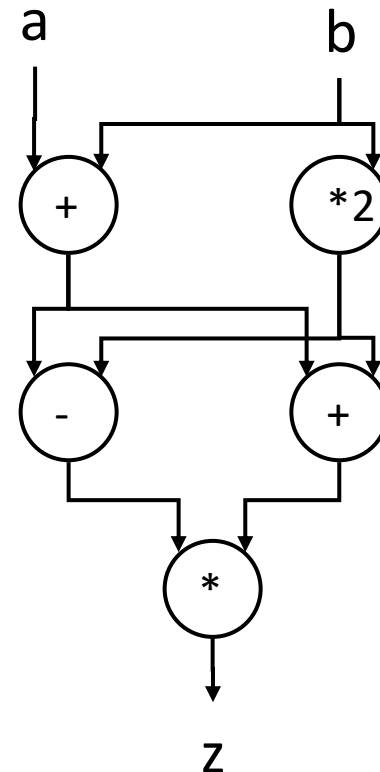
Von Neumann vs. Dataflow

- Consider a Von Neumann program
 - What is the significance of the program order?
 - What is the significance of the storage locations?

**$v = a + b;$
 $w = b * 2;$
 $x = v - w$
 $y = v + w$
 $z = x * y$**

Sequential

**a, b are the only inputs
 z is the only output**

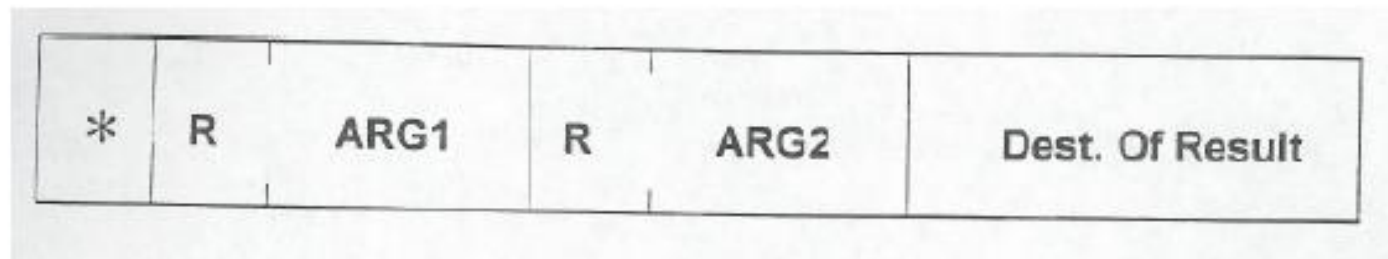
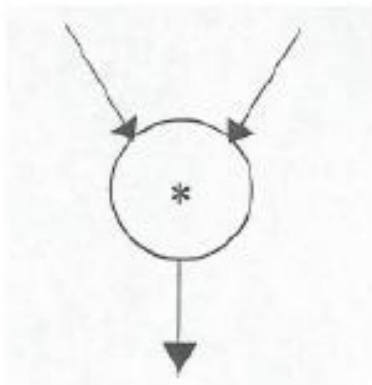


Dataflow

Which model is more natural to you as a programmer?

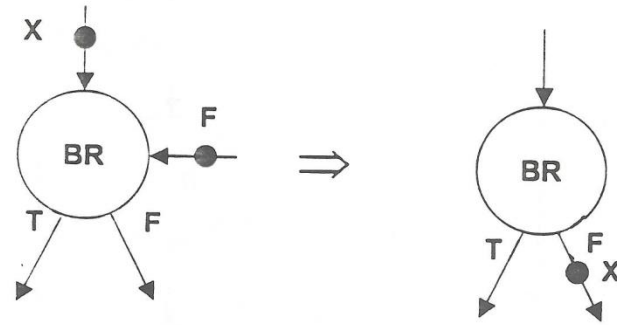
More on Dataflow

- In a dataflow machine, a program consists of dataflow nodes
 - A dataflow node fires (fetched and executed) when all its inputs are ready
 - i.e. when all inputs have tokens
- Dataflow node and its ISA representation

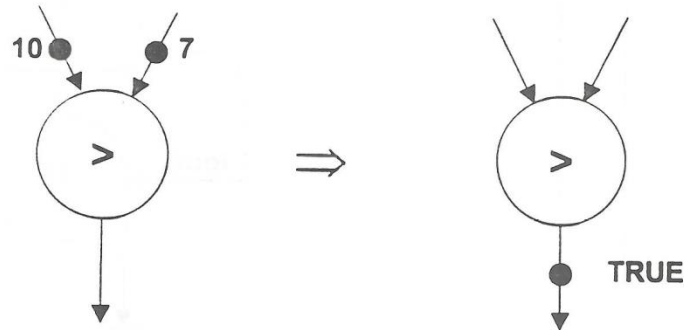


Example Dataflow Nodes

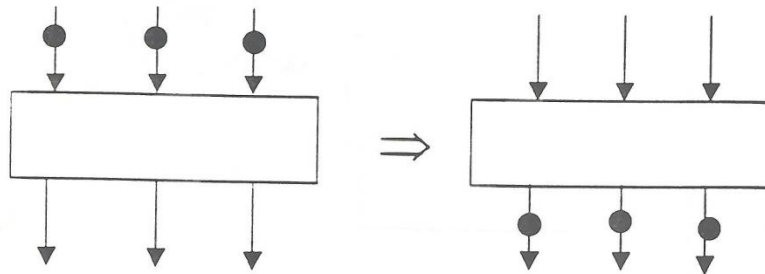
***Conditional**



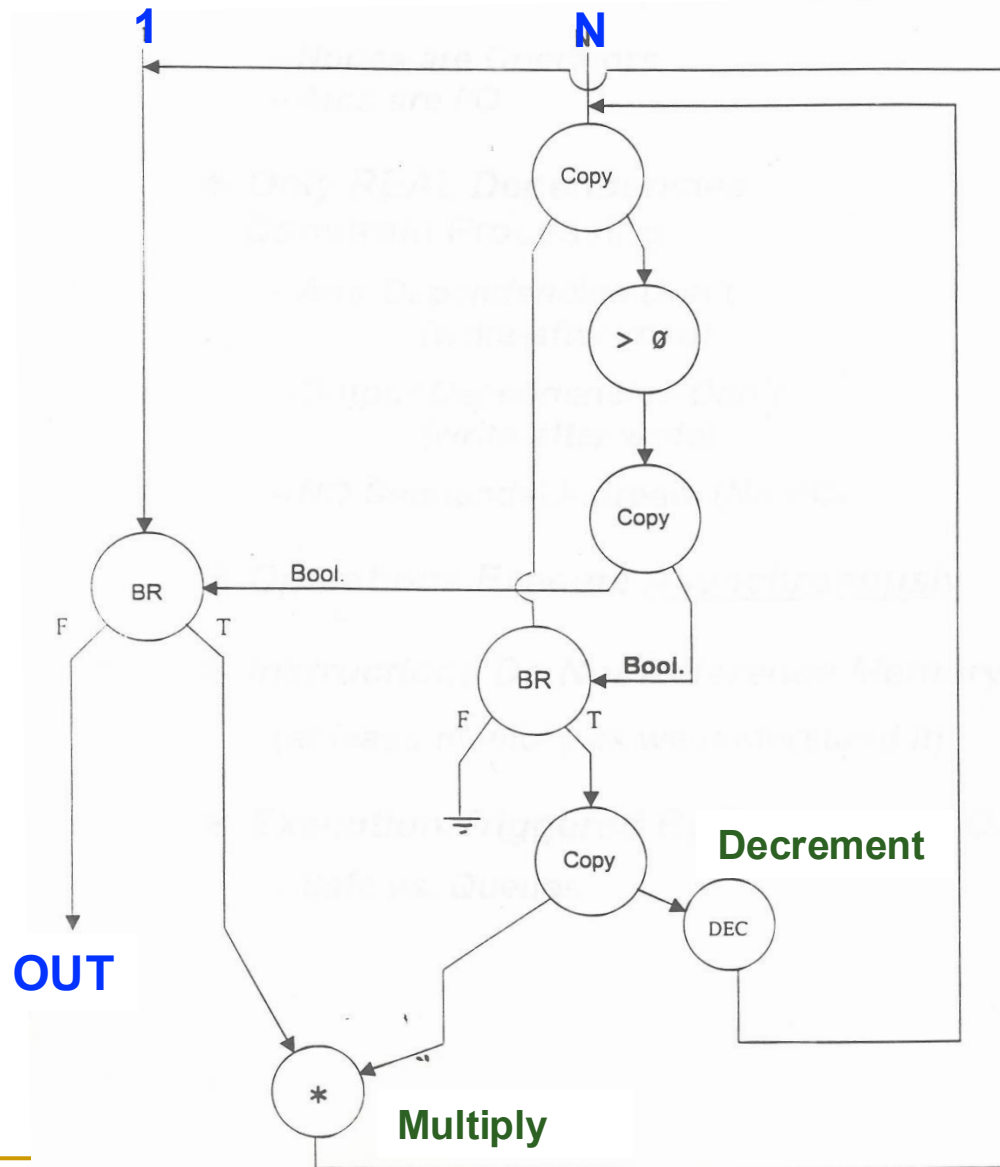
***Relational**



***Barrier Synchron**



A Simple Example Dataflow Program



**N is a
non-negative
integer**

**What is the
value of OUT?**

ISA-level Tradeoff: Program Counter

- Do we want a Program Counter (PC or IP) in the ISA?
 - Yes: Control-driven, sequential execution
 - An instruction is executed when the PC points to it
 - PC automatically changes sequentially (except for control flow instructions) → sequential
 - No: Data-driven, parallel execution
 - An instruction is executed when all its operand values are available → dataflow
- Tradeoffs: MANY high-level ones
 - Ease of programming (for average programmers)?
 - Ease of compilation?
 - Performance: Extraction of parallelism?
 - Hardware complexity?

ISA vs. Microarchitecture Level Tradeoff

- A similar tradeoff (control vs. data-driven execution) can be made at the microarchitecture level
- ISA: Specifies how the **programmer sees** the instructions to be executed
 - Programmer sees a sequential, control-flow execution order vs.
 - Programmer sees a dataflow execution order
- Microarchitecture: How the **underlying implementation actually executes** instructions
 - Microarchitecture can execute instructions in any order as long as it obeys the semantics specified by the ISA when making the instruction results visible to software
 - Programmer should see the order specified by the ISA

Let's Get Back to the von Neumann Model

- But, if you want to learn more about dataflow...
- Dennis and Misunas, "A preliminary architecture for a basic data-flow processor," ISCA 1974.
- Gurd et al., "The Manchester prototype dataflow computer," CACM 1985.
- Some Lecture Videos:
 - <http://www.youtube.com/watch?v=D2uue7izU2c>
 - <http://www.ece.cmu.edu/~ece740/f13/lib/exe/fetch.php?media=onur-740-fall13-module5.2.1-dataflow-part1.ppt>

Lecture Video on Dataflow Architectures

The video content includes two slides titled "Loops and Function Calls Summary".

Figure 11: Interface for a procedure call. On the left, a call of procedure P whose graph is on the right. P has one parameter and one return value. The actual parameter receives a new tag and is sent to the input node of P and concurrently a token containing address A is sent to the output node. This SEND-TO-DESTINATION node transmits the other input token to a node of which the address is contained in the first token. The effect is that, when the return value of the procedure becomes available, the output node sends the result to node A, which returns the tag belonging to the calling expression.

Figure 12: An implementation of a loop using tagged values. At the start of the loop a new tag area is allocated. Tokens belonging to consecutive iterations arrive consecutive tags within the area. The tag from before the loop is inserted on tokens that exit from the loop.

The chalkboard shows the following code and diagrams:

```
while i <= 10
do x = x + 1
y = y * 100
```

Handwritten notes on the chalkboard include:

- $S + A[j] \times B[j]$
- $\langle f, 0, 3, L, \dots \rangle$
- $\langle f, 0, 3, R, 1000 \rangle$

The von Neumann Model

- All major *instruction set architectures* today use this model
 - x86, ARM, MIPS, SPARC, Alpha, POWER, RISC-V, ...
- Underneath (at the microarchitecture level), the execution model of almost all *implementations (or, microarchitectures)* is very different
 - Pipelined instruction execution: *Intel 80486 uarch*
 - Multiple instructions at a time: *Intel Pentium uarch*
 - Out-of-order execution: *Intel Pentium Pro uarch*
 - Separate instruction and data caches
- But, what happens underneath that is **not consistent** with the von Neumann model is **not exposed** to software
 - Difference between ISA and microarchitecture

What is Computer Architecture?

- **ISA+implementation definition:** The science and art of designing, selecting, and interconnecting hardware components and designing the hardware/software interface to create a computing system that meets functional, performance, energy consumption, cost, and other specific goals.
- **Traditional (ISA-only) definition:** “The term *architecture* is used here to describe the attributes of a system as seen by the programmer, i.e., the conceptual structure and functional behavior *as distinct from* the organization of the dataflow and controls, the logic design, and the physical implementation.”

Gene Amdahl, IBM Journal of R&D, April 1964

ISA vs. Microarchitecture

■ ISA

- ❑ Agreed upon interface between software and hardware
 - SW/compiler assumes, HW promises
- ❑ What the software writer needs to know to write and debug system/user programs

■ Microarchitecture

- ❑ Specific implementation of an ISA
- ❑ Not visible to the software

■ Microprocessor

- ❑ **ISA, uarch**, circuits
- ❑ “Architecture” = ISA + microarchitecture

Problem
Algorithm
Program
ISA
Microarchitecture
Circuits
Electrons

Microarchitecture

- A specific **implementation** of the ISA
- How do we implement the ISA?
 - We will discuss this for many lectures
- There can be many implementations of the same ISA
 - **MIPS** R2000, R3000, R4000, R6000, R8000, R10000, ...
 - **x86**: Intel 80486, Pentium, Pentium Pro, Pentium 4, Kaby Lake, Coffee Lake, Comet Lake, Ice Lake, Golden Cove, Sapphire Rapids, ..., AMD K5, K7, K9, Bulldozer, BobCat, Ryzen X, ...
 - **POWER** 4, 5, 6, 7, 8, 9, 10 (IBM), ..., **PowerPC** 604, 605, 620, ...
 - **ARM** Cortex-M*, ARM Cortex-A*, NVIDIA Denver, Apple A*, M1, ...
 - **Alpha** 21064, 21164, 21264, 21364, ...
 - **RISC-V** ...
 - **z/Architecture** ...

ISA vs. Microarchitecture

- What is part of ISA vs. Uarch?
 - Gas pedal: interface for “acceleration”
 - Internals of the engine: implement “acceleration”
- Implementation (uarch) can be various as long as it satisfies the specification (ISA)
 - Add instruction vs. Adder implementation
 - Bit serial, ripple carry, carry lookahead adders are all part of microarchitecture **(see H&H Chapter 5.2.1)**
 - x86 ISA has many implementations:
 - Intel 80486, Pentium, Pentium Pro, Pentium 4, Kaby Lake, Coffee Lake, Comet Lake, Ice Lake, Golden Cover, Sapphire Rapids, ..., AMD K5, K7, K9, Bulldozer, BobCat, Ryzen X, ...
- Microarchitecture usually changes faster than ISA
 - Few ISAs (x86, ARM, SPARC, MIPS, Alpha, RISC-V) but many uarchs
 - *Why?*



ISA: What Does It Specify?

■ Instructions

- ❑ Opcodes, Addressing Modes, Data Types
- ❑ Instruction Types and Formats
- ❑ Registers, Condition Codes

■ Memory

- ❑ Address space, Addressability, Alignment
- ❑ Virtual memory management

■ Call, Interrupt/Exception Handling

■ Access Control, Priority/Privilege

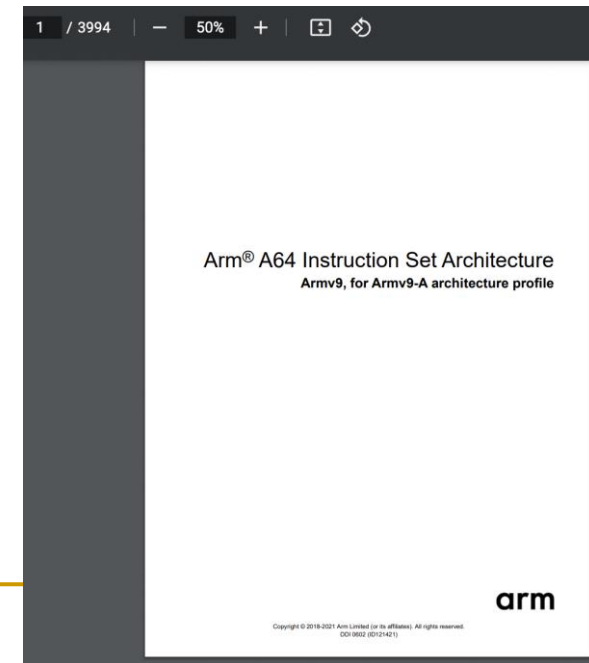
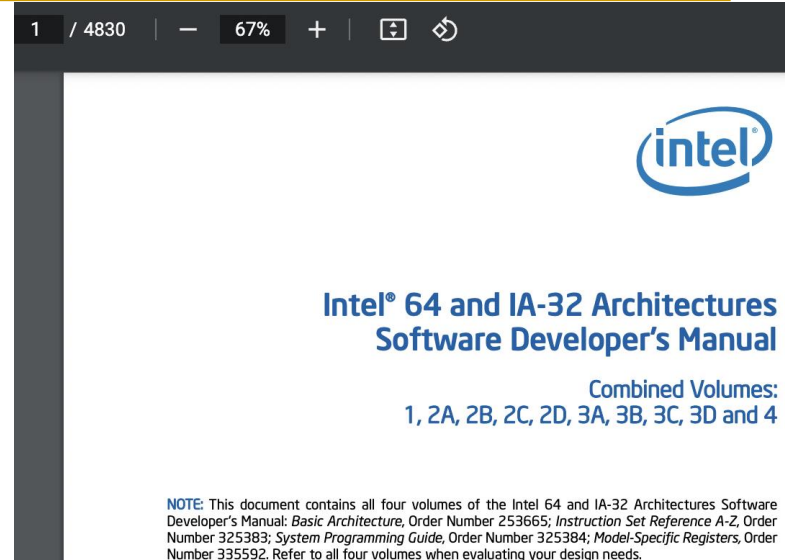
■ I/O: memory-mapped vs. instructions

■ Task/thread Management

■ Power & Thermal Management

■ Multithreading & Multiprocessor support

■ ...



ISAs Keep Getting Extended



ISAs Keep Getting Extended

1 / 5158



100%



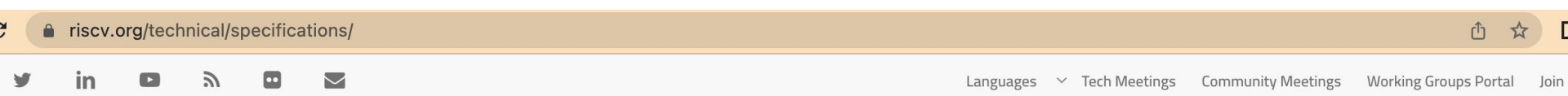
Arm® A64 Instruction Set for A-profile architecture

ISA Manuals: Some Good Bedtime Reading

Combined Volume Set of Intel® 64 and IA-32 Architectures Software Developer's Manuals

Document	Description
Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4	This document contains the following: Volume 1: Describes the architecture and programming environment of processors supporting IA-32 and Intel® 64 architectures. Volume 2: Includes the full instruction set reference, A-Z. Describes the format of the instruction and provides reference pages for instructions. Volume 3: Includes the full system programming guide, parts 1, 2, 3, and 4. Describes the operating-system support environment of Intel® 64 and IA-32 architectures, including: memory management, protection, task management, interrupt and exception handling, multi-processor support, thermal and power management features, debugging, performance monitoring, system management mode, virtual machine extensions (VMX) instructions, Intel® Virtualization Technology (Intel® VT), and Intel® Software Guard Extensions (Intel® SGX). NOTE: Performance monitoring events can be found here: https://perfmon-events.intel.com/ Volume 4: Describes the model-specific registers of processors supporting IA-32 and Intel® 64 architectures.
Intel® 64 and IA-32 Architectures Software Developer's Manual Documentation Changes	Describes bug fixes made to the Intel® 64 and IA-32 architectures software developer's manual between versions. NOTE: This change document applies to all Intel® 64 and IA-32 architectures software developer's manual sets (combined volume set, 4 volume set, and 10 volume set).

ISA Manuals: Some Good Bedtime Reading



[About RISC-V](#) ▾ [Membership](#) ▾ [RISC-V Exchange](#) ▾ [Technical](#) ▾ [News & Events](#) ▾ [Community](#) ▾

Specifications

The RISC-V instruction set architecture (ISA) and related specifications are developed, ratified and maintained by [RISC-V International contributing members](#) within the RISC-V International [Technical Working Groups](#). Work on the specification is [performed on GitHub](#), and the GitHub [issue mechanism](#) can be used to provide input into the specification.

If you would like more information on becoming a member, please see the [membership page](#).

ISA Specification

The specifications shown below represent the current, ratified releases. Work is being done on [GitHub](#).

- Volume 1, Unprivileged Spec v. 20191213 [\[PDF\]](#)
- Volume 2, Privileged Spec v. 20211203 [\[PDF\]](#)
- Recently ratified, but not yet integrated, [extension specifications](#)

Debug Specification

This is the currently ratified specification:

- External Debug Support v. 0.13.2 [\[PDF\]](#) [\[GitHub\]](#)

This is the current stable draft:

- External Debug Support v. 1.0.0-STABLE [\[PDF\]](#)

Trace Specification

The processor trace specification was **approved** on March 20, 2020.

- Trace Specification v. 1.0 [\[PDF\]](#) [\[GitHub\]](#)

Compatibility Test Framework

The RISC-V Architectural Compatibility Test Framework Version 2 is now available. This framework compares arbitrary models against a reference signature, and currently covers RV[32|64]IMC unprivileged specifications only. Tests for the not-yet-ratified Crypto Scalar extension and RV32EMC extensions are also available.

Work on Version 3.0 framework (RISCOF) is

Microarchitecture

- **Implementation of the ISA** under specific **design constraints and goals**
- Anything done in hardware without exposure to software
 - Pipelining
 - In-order versus out-of-order instruction execution
 - Memory access scheduling policy
 - Speculative execution
 - Superscalar processing (multiple instruction issue?)
 - Clock gating
 - Caching? Levels, size, associativity, replacement policy
 - Prefetching?
 - Voltage/frequency scaling?
 - Error correction?

Property of ISA vs. Uarch?

- ADD instruction's opcode
- Type of adder used in the ALU (Bit-serial vs. Ripple-carry)
- Number of general purpose registers
- Number of cycles to execute the MUL instruction
- Number of ports to the register file
- Whether or not the machine employs pipelined instruction execution
- Program counter

- Remember
 - Microarchitecture: **Implementation of the ISA** under specific design constraints and goals

Design Point

- A set of design considerations and their importance
 - **leads to tradeoffs** in both ISA and uarch
- Example considerations:
 - Cost
 - Performance
 - Maximum power consumption, thermal
 - Energy consumption (battery life)
 - Availability
 - Reliability and Correctness
 - Time to Market
 - Security, safety, predictability, ...
- Design point is determined by the “Problem” space (application space), the intended users/ *market*

Problem
Algorithm
Program
ISA
Microarchitecture
Circuits
Electrons

Application Space

Dream, and they will appear...

Other examples of the application space that continue to drive the need for unique design points are the following:

- 1) **scientific applications** such as those whose computations control nuclear power plants, determine where to drill for oil, and predict the weather;
- 2) **transaction-based applications** such as those that handle ATM transfers and e-commerce business;
- 3) **business data processing** applications, such as those that handle inventory control, payrolls, IRS activity, and various personnel record keeping, whether the personnel are employees, students, or voters;
- 4) **network applications**, such as high-speed routing of Internet packets, that enable the connection of your home system to take advantage of the Internet;
- 5) **guaranteed delivery (a.k.a. real time)** applications that require the result of a computation by a certain critical deadline;
- 6) **embedded applications**, where the processor is a component of a larger system that is used to solve the (usually) dedicated application;
- 7) **media applications** such as those that decode video and audio files;
- 8) random software packages that desktop users would like to run on their PCs.

Each of these application areas has a very different set of characteristics. Each application area demands a different set of tradeoffs to be made in specifying the microprocessor to do the job.

Patt, "Requirements, bottlenecks, and good fortune: agents for microprocessor evolution,"
Proc. of the IEEE 2001.

Many other workloads:

Genome analysis

Machine learning

Robotics

Web search

Graph analytics

...

Increasingly Demanding Applications

Dream

and, they will come

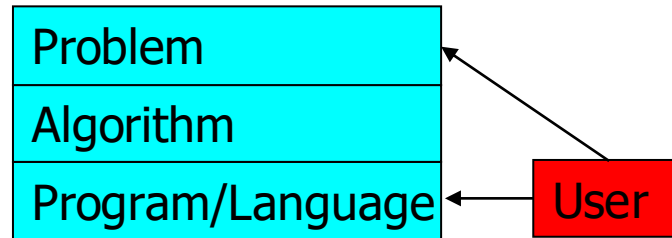
As applications push boundaries, computing platforms will become increasingly strained.

Tradeoffs: Soul of Computer Architecture

- ISA-level tradeoffs
- Microarchitecture-level tradeoffs
- System and Task-level tradeoffs
 - How to divide the labor between hardware and software
- *Computer architecture is the science and art of making the appropriate trade-offs to meet a design point*
 - *Why **art**?*

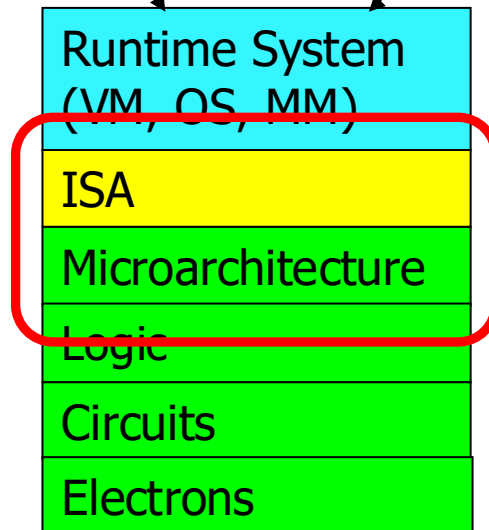
Why Is It (Somewhat) Art?

New demands
from the top
(Look Up)



New demands and
personalities of users
(Look Up)

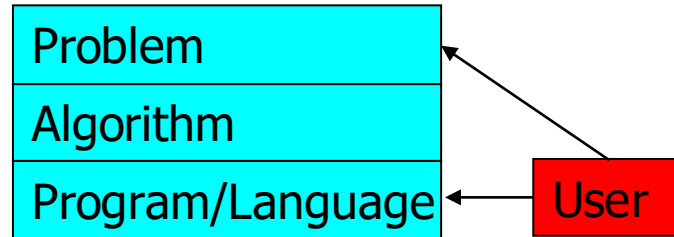
New issues and
capabilities
at the bottom
(Look Down)



- We do not (fully) know the future (applications, users, market)

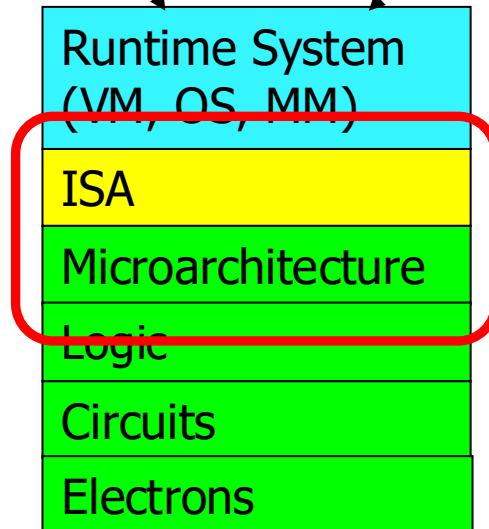
Why Is It (Somewhat) Art?

Changing demands
at the top
(Look Up and Forward)



Changing demands and
personalities of users
(Look Up and Forward)

Changing issues and
capabilities
at the bottom
(Look Down and Forward)



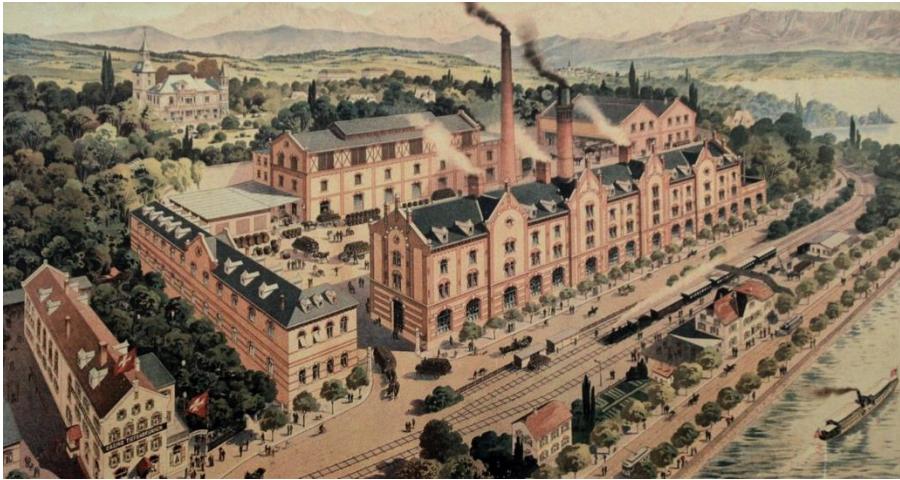
- And, the future is not constant (it changes)!

Analogue from Macro-Architecture

- Future is not constant in macro-architecture, either
- Example: Can a mill be later used as a theater + restaurant + conference room?

Mühle Tiefenbrunnen in Zurich

- Originally built as a **brewery** in 1889
 - part of it was converted into a **mill** in 1913
 - and the other part into a **cold store**
- Today is a **center for a variety of activities**: theater, conferences, restaurants, shops, museum...



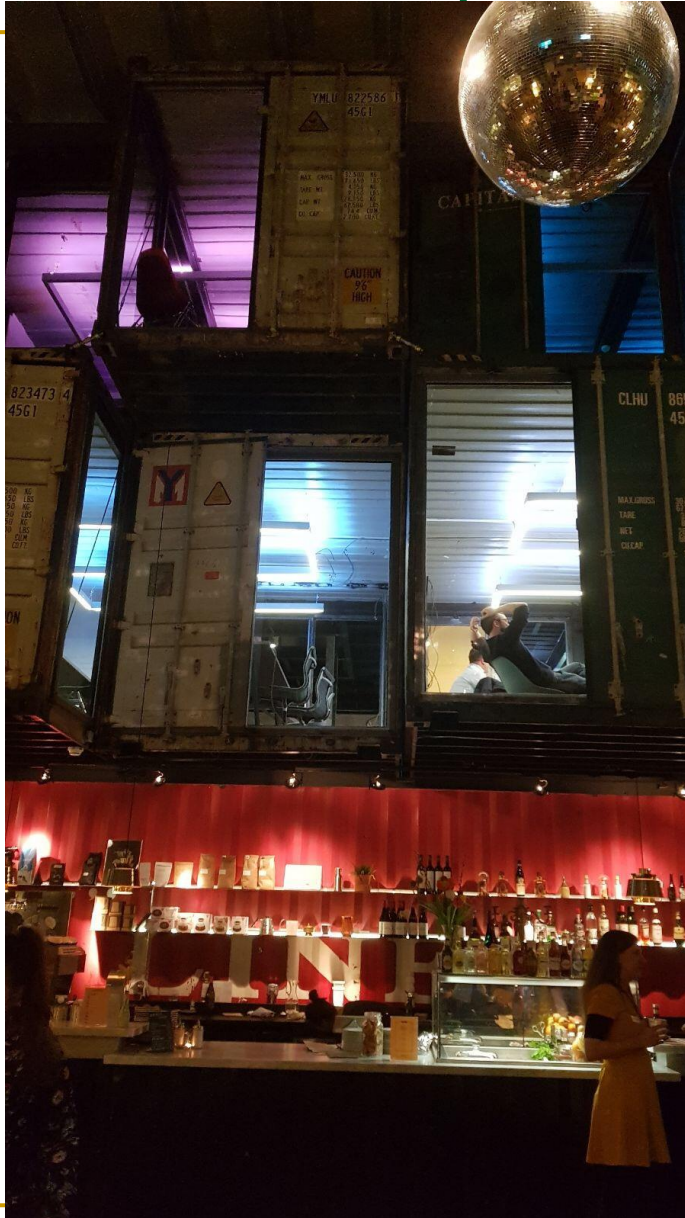
Brewery in 1900



Another Example in Zurich (I)



Another Example in Zurich (II)





By Roland zh (Own work) [CC BY-SA 3.0
(<https://creativecommons.org/licenses/by-sa/3.0/>)],
via Wikimedia Commons

Yet Another Example from Pittsburgh (I)



Yet Another Example from Pittsburgh (II)

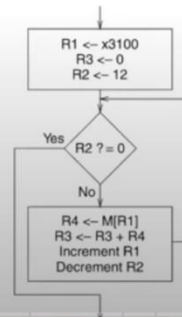


Extra Lecture on Assembly Programming

A Program for Adding Integers in LC-3

- We use conditional branch instructions to create a loop

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	LEA	1	1	0	0	0	1	0	x00FF	1	1	1	1	1	1	1	R1 = PC + 0x00FF = 3100 // load address
x3001	AND	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 = 0 // reset register
x3002	AND	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 = 0 // reset register
x3003	ADD	0	0	1	0	1	0	0	1	0	1	0	1	1	0	0	R2 = R2 + 12 // initialize counter
x3004	BR	0	0	0	0	2	0	5	0	0	0	0	0	1	0	1	BRz (PC + 5) = BRz 0x300A // check condition
x3005	LDR	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	R4 = M[R1 + 0] // load value
x3006	ADD	0	0	1	0	1	1	0	1	1	0	0	0	1	0	0	R3 = R3 + R4 // accumulate
x3007	ADD	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	R1 = R1 + 1 // increment address
x3008	ADD	0	0	1	0	1	0	0	1	0	1	1	1	1	1	1	R2 = R2 - 1 // decrement counter
x3009	BR	0	0	0	n	z	p	6	1	1	1	1	1	0	1	0	BRnzp (PC + 6) = BRnzp 0x3004 // jump



Digital Design & Computer Architecture - Lecture 10b: Assembly Programming (Spring 2022)



Onur Mutlu Lectures
31.8K subscribers

Analytics

Edit video

46



Share

Download

Clip

Save



2K views 11 months ago Livestream - Digital Design and Computer Architecture - ETH Zürich (Spring 2022)

Digital Design and Computer Architecture, ETH Zürich, Spring 2022 (<https://safari.ethz.ch/digitaltechnik...>)

Extra Lecture on Assembly Programming

Function Calls: Code Example

High-level code

```
int main()
{
    int y;
    ...
    // 4 arguments
    y = diffofsums(2, 3, 4, 5);
    ...
}

int diffofsums(int f, int g,
               int h, int i)
{
    int result;
    result = (f + g) - (h + i);
    // return value
    return result;
}
```

MIPS assembly

```
# $s0 = y
main:
...
addi $a0, $0, 2    # argument 0 = 2
addi $a1, $0, 3    # argument 1 = 3
addi $a2, $0, 4    # argument 2 = 4
addi $a3, $0, 5    # argument 3 = 5
jal  diffofsums    # call procedure
add  $s0, $v0, $0   # y = returned value
...

# $s0 = result
diffofsums:
add  $t0, $a0, $a1  # $t0 = f + g
add  $t1, $a2, $a3  # $t1 = h + i
sub  $s0, $t0, $t1  # result=(f + g) - (h + i)
add  $v0, $s0, $0   # put return value in $v0
jr   $ra           # return to caller
```

Argument values

Return value

Return address

Digital Design & Computer Architecture - Lecture 10b: Assembly Programming (Spring 2022)



Onur Mutlu Lectures
31.8K subscribers

Analytics

Edit video

46



Share

Download

Clip

Save



2,069 views Premiered Mar 28, 2022 Livestream - Digital Design and Computer Architecture - ETH Zürich (Spring 2022)
Digital Design and Computer Architecture, ETH Zürich, Spring 2022 (<https://safari.ethz.ch/digitaltechnik...>)

Digital Design & Computer Arch.

Lecture 9a: ISA & Microarchitecture

Prof. Onur Mutlu

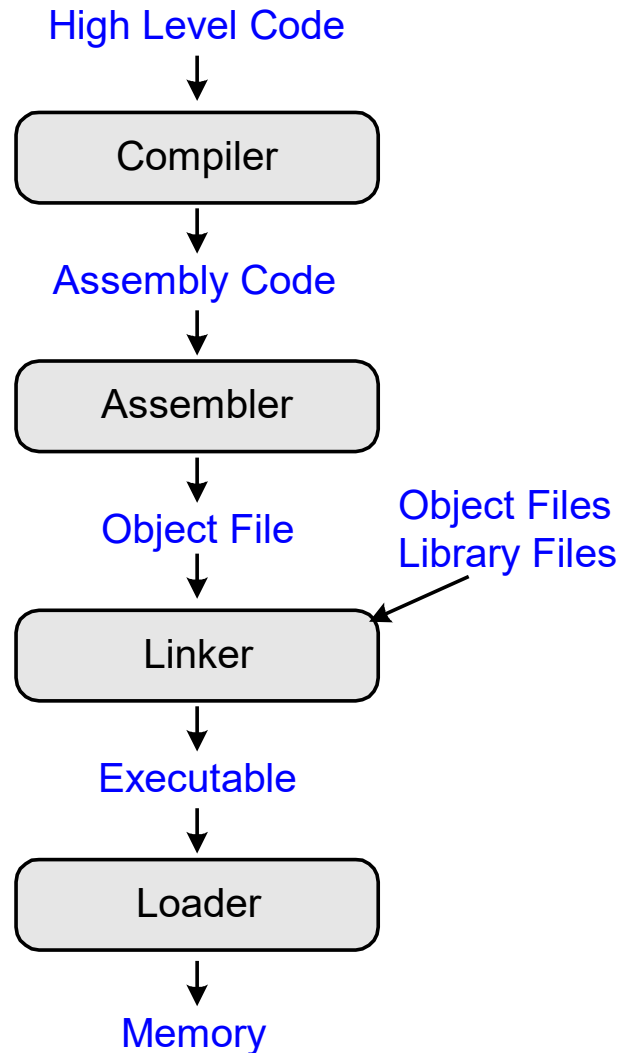
ETH Zürich

Spring 2026

19 March 2026

Additional Slides

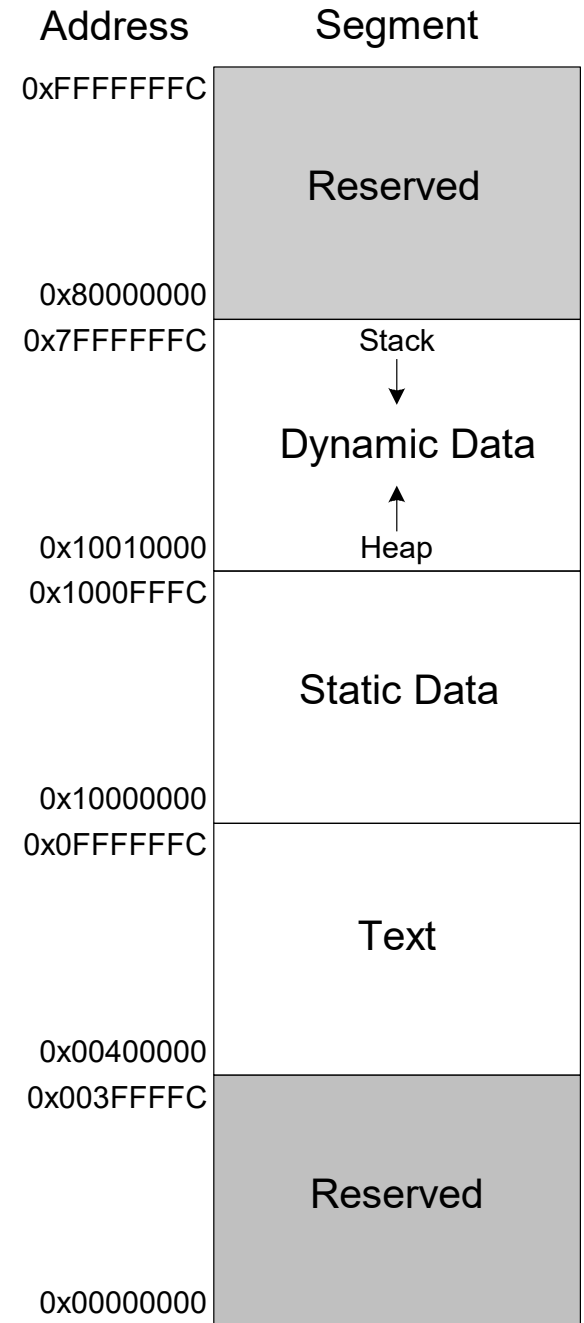
How Do We Compile & Run an Application?



What needs to be stored in memory?

- **Instructions (also called text)**
- **Data**
 - Global/static: allocated before program begins
 - Dynamic: allocated within program
- **How big is memory?**
 - At most $2^{32} = 4$ gigabytes (4 GB)
 - From address `0x00000000` to `0xFFFFFFFF`

The MIPS Memory Map



Example Program: C Code

```
int f, g, y; // global variables

int main(void)
{
    f = 2;
    g = 3;
    y = sum(f, g);

    return y;
}

int sum(int a, int b) {
    return (a + b);
}
```

Example Program: Assembly Code

```
int f, g, y; // global
```

```
int main(void)
{

    f = 2;

    g = 3;

    y = sum(f, g);
    return y;
}
```

```
int sum(int a, int b) {
    return (a + b);
}
```

```
.data
```

```
f:
```

```
g:
```

```
y:
```

```
.text
```

```
main: addi $sp, $sp, -4 # stack
      sw   $ra, 0($sp)  # store $ra
      addi $a0, $0, 2   # $a0 = 2
      sw   $a0, f       # f = 2
      addi $a1, $0, 3   # $a1 = 3
      sw   $a1, g       # g = 3
      jal  sum          # call sum
      sw   $v0, y       # y = sum()
      lw   $ra, 0($sp)  # rest. $ra
      addi $sp, $sp, 4  # rest. $sp
      jr   $ra          # return
sum:   add  $v0, $a0, $a1 # $v0= a+b
      jr   $ra          # return
```

Example Program: Symbol Table

Symbol	Address
f	0x10000000
g	0x10000004
y	0x10000008
main	0x00400000
sum	0x0040002C

Example Program: Executable

Executable file header	Text Size	Data Size
	0x34 (52 bytes)	0xC (12 bytes)
Text segment	Address	Instruction
	0x00400000	0x23BDFFFC
	0x00400004	0xAFBF0000
	0x00400008	0x20040002
	0x0040000C	0xAF848000
	0x00400010	0x20050003
	0x00400014	0xAF858004
	0x00400018	0x0C10000B
	0x0040001C	0xAF828008
	0x00400020	0x8FBF0000
	0x00400024	0x23BD0004
	0x00400028	0x03E00008
	0x0040002C	0x00851020
	0x00400030	0x03E00008
Data segment	Address	Data
	0x10000000	f
	0x10000004	g
	0x10000008	y

```

addi $sp, $sp, -4
sw   $ra, 0($sp)
addi $a0, $0, 2
sw   $a0, 0x8000($gp)
addi $a1, $0, 3
sw   $a1, 0x8004($gp)
jal  0x0040002C
sw   $v0, 0x8008($gp)
lw   $ra, 0($sp)
addi $sp, $sp, 4
jr   $ra
add  $v0, $a0, $a1
jr   $ra

```

Example Program: In Memory

